

**LAPORAN PRAKTIKUM
PEMROGRAMAN MOBILE
MODUL 4**



VIEWMODEL DAN DEBUGGING

Oleh:

Arya Arrozza Ridho Syaputra

NIM. 2410817210010

**PROGRAM STUDI TEKNOLOGI INFORMASI
FAKULTAS TEKNIK
UNIVERSITAS LAMBUNG MANGKURAT
MEI 2026**

LEMBAR PENGESAHAN
LAPORAN PRAKTIKUM PEMROGRAMAN MOBILE
MODUL 4

Laporan Praktikum Pemrograman Mobile Modul 4: ViewModel dan Debugging Sederhana ini disusun sebagai syarat lulus mata kuliah Praktikum Pemrograman Mobile. Laporan Praktikum ini dikerjakan oleh:

Nama Praktikan : Arya Arrozza Ridho Syaputra
NIM : 2410817210010

Menyetujui,
Asisten Praktikum

Mengetahui,
Dosen Penanggung Jawab Praktikum

Muhammad Erza Raihan
NIM. 2310817210027

Irham Maulani Abdul Gani, S.Kom.,
M.Kom.
NIP. 199710312025061009

DAFTAR ISI

LEMBAR PENGESAHAN	2
DAFTAR ISI.....	3
DAFTAR GAMBAR	4
DAFTAR TABEL	4
SOAL	6
A. Source Code	7
B. Output Program.....	50
C. Pembahasan.....	54
D. Essay	66
E. Daftar Pustaka	67
Tautan GIT	68

DAFTAR GAMBAR

Gambar 1. Screenshot Hasil Log Loading Data XML.....	50
Gambar 2. Screenshot Hasil Log Tombol Detail dan List Data XML.....	50
Gambar 3. Screenshot Hasil Log Tombol Intent XML.....	51
Gambar 4. Screenshot Hasil Debugging XML	51
Gambar 5. Screenshot Hasil Log Loading Data Compose	52
Gambar 6. Screenshot Hasil Log Tombol Detail dan List Data Compose	52
Gambar 7. Screenshot Hasil Log Tombol Intent Compose	53
Gambar 8. Screenshot Hasil Debugging Compose	53

DAFTAR TABEL

Tabel 1. Source Code MainActivity.Kt XML.....	7
Tabel 2. Source Code ScrollableViewModel.Kt XML	7
Tabel 3. Source Code ScrollableUiState.Kt XML	9
Tabel 4. Source Code ScrollableData.Kt XML	9
Tabel 5. Source Code DataSource.Kt XML.....	10
Tabel 6. Source Code SettingsFragment.Kt XML	10
Tabel 7. Source Code DetailFragment.Kt XML	12
Tabel 8. Source Code HomeFragment.Kt XML	14
Tabel 9. Source Code CarouselAdapter.Kt XML	16
Tabel 10. Source Code CarouselHeaderAdapter.Kt XML.....	17
Tabel 11. Source Code ItemCardAdapter.Kt XML	18
Tabel 12. Source Code CarouselViewHolder.Kt XML	19
Tabel 13. Source Code ItemCardViewHolder.Kt XML	19
Tabel 14. Source Code navigation_graph.xml.....	20
Tabel 15. Source Code activity_main.xml.....	21
Tabel 16. Source Code fragment_detail.xml.....	21
Tabel 17. Source Code fragment_settings.xml	24
Tabel 18. Source Code fragment_home.xml.....	26

Tabel 19. Source Code menu_homel.xml	27
Tabel 20. Source Code home_carousel_header.xml	27
Tabel 21. Source Code home_carousel_card.xml	27
Tabel 22. Source Code home_item_card.xml	28
Tabel 23. Source Code MainActivity.Kt Compose	30
Tabel 24. Source Code ScrollableApp.Kt Compose	31
Tabel 25. Source Code ScrollableData.Kt Compose	33
Tabel 26. Source Code DataSource.Kt Compose	33
Tabel 27. Source Code ScrollableViewModel.Kt Compose.....	34
Tabel 28. Source Code ScrollableUiState.Kt Compose.....	36
Tabel 29. Source Code DetailScreen.Kt Compose	36
Tabel 30. Source Code SettingScreen.Kt Compose.....	39
Tabel 31. Source Code HomeScreen.Kt Compose	41

SOAL

Soal Praktikum:

Lanjutkan aplikasi Android berbasis XML dan Jetpack Compose yang sudah dibuat pada Modul 3 dengan menambahkan modifikasi sesuai ketentuan berikut:

1. Buatlah sebuah ViewModel untuk menyimpan dan mengelola data dari list item. Data tidak boleh disimpan langsung di dalam Fragment atau Activity.
2. Gunakan ViewModelFactory untuk membuat parameter dengan tipe data String di dalam ViewModel
3. Gunakan StateFlow untuk mengelola event onClick dan data list item dari ViewModel ke Fragment
4. Install dan gunakan library Timber untuk logging event berikut:
 - a. Log saat data item masuk ke dalam list
 - b. Log saat tombol Detail dan tombol Explicit Intent ditekan
 - c. Log data dari list yang dipilih ketika berpindah ke halaman Detail
 - d. Gunakan tool Debugger di Android Studio untuk melakukan debugging pada aplikasi.

Cari setidaknya satu breakpoint yang relevan dengan aplikasi. Lalu, gunakan fitur Step Into, Step Over, dan Step Out. Setelah itu, jelaskan fungsi Debugger, cara menggunakan Debugger, serta fitur Step Into, Step Over, dan Step Out

A. Source Code

Tabel 1. Source Code MainActivity.Kt XML

1	package com.example.scrollablexml
2	
3	import android.os.Bundle
4	import androidx.activity.enableEdgeToEdge
5	import androidx.activity.viewModels
6	import androidx.appcompat.app.AppCompatActivity
7	import androidx.appcompat.app.AppCompatActivity
8	import androidx.core.view.WindowCompat
9	import
	com.example.scrollablexml.databinding.ActivityMainBinding
10	import com.example.scrollablexml.ui.ScrollableViewModel
11	import timber.log.Timber
12	
13	class MainActivity : AppCompatActivity() {
14	private lateinit var binding: ActivityMainBinding
15	private val viewModel: ScrollableViewModel by
	viewModels {
16	val locale =
	AppCompatActivity.getApplicationLocales().get(0)?.language
	?: "en"
17	ScrollableViewModel.Factory(locale)
18	}
19	
20	override fun onCreate(savedInstanceState: Bundle?) {
21	super.onCreate(savedInstanceState)
22	Timber.plant(Timber.DebugTree())
23	enableEdgeToEdge()
24	WindowCompat.setDecorFitsSystemWindows(window,
	false)
25	binding =
	ActivityMainBinding.inflate(layoutInflater)
26	setContentView(binding.root)
27	viewModel
28	}
29	}

Tabel 2. Source Code ScrollableViewModel.Kt XML

1	package com.example.scrollablexml.ui
2	
3	import androidx.appcompat.app.AppCompatActivity
4	import androidx.core.os.LocaleListCompat
5	import androidx.lifecycle.ViewModel
6	import androidx.lifecycle.ViewModelProvider
7	import androidx.lifecycle.viewmodel.initializer

```

8 import androidx.lifecycle.viewmodel.viewModelFactory
9 import com.example.scrollablexml.model.DataSource
10 import kotlinx.coroutines.flow.MutableStateFlow
11 import kotlinx.coroutines.flow.StateFlow
12 import kotlinx.coroutines.flow.asStateFlow
13 import timber.log.Timber
14
15 class ScrollableViewModel(initialLocale: String) :
16     ViewModel() {
17         private val _uiState =
18             MutableStateFlow(ScrollableUiState())
19         val uiState: StateFlow<ScrollableUiState> =
20             _uiState.asStateFlow()
21
22         init {
23             val items = DataSource().loadItems()
24             Timber.d("Items loaded: ${items.size}")
25             Timber.d("Items: $items")
26             _uiState.value = ScrollableUiState(
27                 list = items,
28                 currentItemIndex = 0,
29                 selectedLocale = initialLocale
30             )
31         }
32
33         fun updateCurrentItem(index: Int) {
34             val selectedItem =
35                 _uiState.value.list.getOrNull(index)
36             Timber.d("Selected at index $index of itemId:
37                 ${selectedItem?.id}")
38             Timber.d("Selected items: $selectedItem")
39
40             _uiState.value = _uiState.value.copy(
41                 currentItemIndex = index
42             )
43         }
44
45         fun onDetailButtonClicked() {
46             Timber.d("Detail button clicked")
47         }
48
49         fun onIntentButtonClicked() {
50             Timber.d("Intent button clicked")
51         }
52
53         fun updateLocale(locale: String) {

```


49	<code>_uiState.value = _uiState.value.copy(</code>
50	<code>selectedLocale = locale,</code>
51	<code>)</code>
52	<code>val appLocale =</code>
	<code>LocaleListCompat.forLanguageTags(locale)</code>
53	<code>AppCompatDelegate.setApplicationLocales(appLocale)</code>
54	<code>}</code>
55	
56	<code>companion object {</code>
57	<code>fun Factory(initialLocale: String):</code>
	<code>ViewModelProvider.Factory = viewModelFactory {</code>
58	<code>initializer {</code>
59	<code>ScrollableViewModel(initialLocale)</code>
60	<code>}</code>
61	<code>}</code>
62	<code>}</code>
63	<code>}</code>

Tabel 3. Source Code ScrollableUiState.Kt XML

1	<code>package com.example.scrollablexml.ui</code>
2	
3	<code>import com.example.scrollablexml.model.ScrollableData</code>
4	
5	<code>data class ScrollableUiState(</code>
6	<code>val list: List<ScrollableData> = emptyList(),</code>
7	<code>val currentItemIndex: Int = 0,</code>
8	<code>val selectedLocale: String = "en"</code>
9	<code>)</code>

Tabel 4. Source Code ScrollableData.Kt XML

1	<code>package com.example.scrollablexml.model</code>
2	
3	<code>import androidx.annotation.DrawableRes</code>
4	<code>import androidx.annotation.StringRes</code>
5	
6	<code>data class ScrollableData(</code>
7	<code>val id: Int,</code>
8	<code>@StringRes val titleResourceId: Int,</code>
9	<code>@StringRes val subtitleResourceId: Int,</code>
10	<code>@StringRes val descriptionResourceId: Int,</code>
11	<code>@StringRes val detailResourceId: Int,</code>
12	<code>@DrawableRes val imageResourceId: Int,</code>
13	<code>@DrawableRes val detailImageResourceId: Int,</code>
14	<code>val steamUrl: String</code>
15	<code>)</code>

Tabel 5. Source Code DataSource.Kt XML

1	package com.example.scrollablexml.model
2	
3	import com.example.scrollablexml.R
4	
5	class DataSource {
6	fun loadItems(): List<ScrollableData> {
7	return listOf(
8	ScrollableData(1, R.string.item1,
	R.string.item1_sub, R.string.item1_desc,
	R.string.item1_detail, R.drawable.crosscode,
	R.drawable.crosscode_detail,
	"https://store.steampowered.com/app/368340/CrossCode/"),
9	ScrollableData(2, R.string.item2,
	R.string.item2_sub, R.string.item2_desc,
	R.string.item2_detail, R.drawable.hades2,
	R.drawable.hades2_detail,
	"https://store.steampowered.com/app/1145350/Hades_II/"),
10	ScrollableData(3, R.string.item3,
	R.string.item3_sub, R.string.item3_desc,
	R.string.item3_detail, R.drawable.nms,
	R.drawable.nms_detail,
	"https://store.steampowered.com/app/275850/No_Mans_Sky/"
	),
11	ScrollableData(4, R.string.item4,
	R.string.item4_sub, R.string.item4_desc,
	R.string.item4_detail, R.drawable.coe33,
	R.drawable.coe33_detail,
	"https://store.steampowered.com/app/1903340/Clair_Obscur
	_Expedition_33/"),
12	ScrollableData(5, R.string.item5,
	R.string.item5_sub, R.string.item5_desc,
	R.string.item5_detail, R.drawable.sekiro,
	R.drawable.sekiro_detail,
	"https://store.steampowered.com/app/814380/Sekiro_Shadow
	s_Die_Twice__GOTY_Edition/")
13	)
14	}
15	}

Tabel 6. Source Code SettingsFragment.Kt XML

1	package com.example.scrollablexml.ui.settings
2	
3	import android.os.Bundle
4	import androidx.fragment.app.Fragment
5	import android.view.LayoutInflater

```

6 import android.view.View
7 import android.view.ViewGroup
8 import androidx.fragment.app.activityViewModels
9 import androidx.lifecycle.Lifecycle
10 import androidx.lifecycle.LifecycleScope
11 import androidx.lifecycle.repeatOnLifecycle
12 import androidx.navigation.fragment.findNavController
13 import com.example.scrollablexml.R
14 import
15     com.example.scrollablexml.databinding.FragmentSettingsBi
16     nding
17 import com.example.scrollablexml.ui.ScrollableViewModel
18 import kotlinx.coroutines.launch
19
20 class SettingsFragment : Fragment() {
21     private var _binding: FragmentSettingsBinding? =
22     null
23     private val binding get() = _binding!!
24     private val viewModel: ScrollableViewModel by
25     activityViewModels()
26
27     override fun onCreateView(
28         inflater: LayoutInflater, container: ViewGroup?,
29         savedInstanceState: Bundle?
30     ): View {
31         _binding =
32         FragmentSettingsBinding.inflate(inflater, container,
33         false)
34         return binding.root
35     }
36
37     override fun onViewCreated(view: View,
38         savedInstanceState: Bundle?) {
39         super.onViewCreated(view, savedInstanceState)
40
41         binding.toolbar.setNavigationOnClickListener {
42             findNavController().navigateUp()
43         }
44
45         viewLifecycleOwner.lifecycleScope.launch {
46             viewLifecycleOwner.repeatOnLifecycle(Lifecycle.State.STA
47             RTED) {
48                 viewModel.uiState.collect { uiState ->

```

42	val checkedLocaleIn = if
	(uiState.selectedLocale == "en")
43	R.id.rb_english else
	R.id.rb_indonesian
44	if
	(binding.rgLocale.checkedRadioButtonId !=
	checkedLocaleIn)
45	binding.rgLocale.check(checkedLocaleIn)
46	}
47	}
48	}
49	
50	binding.rgLocale.setOnCheckedChangeListener { _,
	checkedLocaleIn ->
51	val locale = if (checkedLocaleIn ==
	R.id.rb_english) "en" else "id"
52	viewModel.updateLocale(locale)
53	}
54	}
55	
56	override fun onDestroyView() {
57	super.onDestroyView()
58	_binding = null
59	}
60	}

Tabel 7. Source Code DetailFragment.Kt XML

1	package com.example.scrollablexml.ui.detail
2	
3	import android.os.Bundle
4	import androidx.fragment.app.Fragment
5	import android.view.LayoutInflater
6	import android.view.View
7	import android.view.ViewGroup
8	import androidx.fragment.app.activityViewModels
9	import androidx.lifecycle.Lifecycle
10	import androidx.lifecycle.LifecycleScope
11	import androidx.lifecycle.repeatOnLifecycle
12	import androidx.navigation.fragment.findNavController
13	import androidx.navigation.fragment.navArgs
14	import
	com.example.scrollablexml.databinding.FragmentDetailBind
	ing
15	import com.example.scrollablexml.ui.ScrollableViewModel
16	import kotlinx.coroutines.launch

```

17
18 class DetailFragment : Fragment() {
19     private var _binding: FragmentDetailBinding? = null
20     private val binding get() = _binding!!
21     private val viewModel: ScrollableViewModel by
activityViewModels()
22     private val args: DetailFragmentArgs by navArgs()
23
24     override fun onCreateView(
25         inflater: LayoutInflater,
26         container: ViewGroup?,
27         savedInstanceState: Bundle?
28     ): View {
29         _binding =
FragmentDetailBinding.inflate(inflater, container,
false)
30         return binding.root
31     }
32
33     override fun onViewCreated(view: View,
savedInstanceState: Bundle?) {
34         super.onViewCreated(view, savedInstanceState)
35
36         binding.toolbar.setNavigationOnClickListener {
37             findNavController().navigateUp()
38         }
39
40         val index = args.itemIndex
41         viewModel.updateCurrentItem(index)
42
43         viewLifecycleOwner.lifecycleScope.launch {
44
viewLifecycleOwner.repeatOnLifecycle(Lifecycle.State.STA
RTED) {
45             viewModel.uiState.collect { uiState ->
46                 val item =
uiState.list.getOrNull(index) ?: uiState.list.first()
47                 binding.apply {
48
toolbar.setTitle(item.titleResourceId)
49
ivDetail.setImageResource(item.imageResourceId)
50
tvTitle.setText(item.titleResourceId)
51
tvSubtitle.setText(item.subtitleResourceId)

```

52	tvDesc.setText(item.descriptionResourceId)
53	tvDetail.setText(item.detailResourceId)
54	}
55	}
56	}
57	}
58	
59	
60	}
61	
62	override fun onDestroyView() {
63	super.onDestroyView()
64	_binding = null
65	}
66	}

Tabel 8. Source Code HomeFragment.Kt XML

1	package com.example.scrollablexml.ui.home
2	
3	import android.content.Intent
4	import androidx.recyclerview.widget.ConcatAdapter
5	import android.os.Bundle
6	import androidx.fragment.app.Fragment
7	import android.view.LayoutInflater
8	import android.view.View
9	import android.view.ViewGroup
10	import androidx.fragment.app.activityViewModels
11	import androidx.lifecycle.Lifecycle
12	import androidx.lifecycle.LifecycleScope
13	import androidx.lifecycle.repeatOnLifecycle
14	import androidx.navigation.fragment.findNavController
15	import com.example.scrollablexml.R
16	import com.example.scrollablexml.databinding. FragmentHomeBinding
17	import com.example.scrollablexml.ui.ScrollableViewModel
18	import com.example.scrollablexml.ui.home. adapter.CarouselHeaderAdapter
19	import com.example.scrollablexml.ui.home. adapter.ItemCardAdapter
20	import kotlinx.coroutines.launch
21	import androidx.core.net.toUri
22	import androidx.recyclerview.widget.LinearLayoutManager
23	
24	

```

25
26 class HomeFragment : Fragment() {
27     private var _binding: FragmentHomeBinding? = null
28     private val binding get() = _binding!!
29     private val viewModel: ScrollableViewModel by
activityViewModels()
30
31     override fun onCreateView(
32         inflater: LayoutInflater, container: ViewGroup?,
33         savedInstanceState: Bundle?
34     ): View {
35         _binding = FragmentHomeBinding.inflate(inflater,
container, false)
36         return binding.root
37     }
38
39     override fun onViewCreated(view: View,
savedInstanceState: Bundle?) {
40         super.onViewCreated(view, savedInstanceState)
41
42         binding.toolbar.inflateMenu(R.menu.menu_home)
43         binding.toolbar.setOnMenuItemClickListener {
item ->
44             when (item.itemId) {
45                 R.id.action_setting -> {
46
findNavController().navigate(R.id.action_homeFragment_to
_settingsFragment)
47                     true
48                 }
49                 else -> false
50             }
51         }
52         binding.toolbar.setNavigationOnClickListener {
53             requireActivity().finish()
54         }
55
56         viewLifecycleOwner.lifecycleScope.launch {
57             viewLifecycleOwner.repeatOnLifecycle(
Lifecycle.State.STARTED) {
58                 viewModel.uiState.collect { uiState ->
59                     val carouselAdapter =
CarouselHeaderAdapter(
60                         items = uiState.list,
61                         onDetailClick = { index ->

```

62	val action =
	HomeFragmentDirections
	.actionHomeFragmentToDetailFragment(itemIndex = index)
63	findNavController().navigate(action)
64	}
65	)
66	val itemCardAdapter =
	ItemCardAdapter(
67	uiState.list,
68	onDetailClicked = { index ->
69	viewModel.onDetailButtonClicked()
70	val action =
	HomeFragmentDirections
	.actionHomeFragmentToDetailFragment(itemIndex = index)
71	findNavController().navigate(action)
72	},
73	onSteamClicked = { url ->
74	viewModel.onIntentButtonClicked()
75	startActivity(Intent(Intent.ACTION_VIEW, url.toUri()))
76	}
77	)
78	binding.rvMain.apply {
79	layoutManager =
	LinearLayoutManager(requireContext())
80	adapter =
	ConcatAdapter(carouselAdapter, itemCardAdapter)
81	}
82	}
83	}
84	}
85	}
86	
87	override fun onDestroyView() {
88	super.onDestroyView()
89	_binding = null
90	}
91	}

Tabel 9. Source Code CarouselAdapter.Kt XML

1	package com.example.scrollablexml.ui.home.adapter
2	

3	import android.view.LayoutInflater
4	import android.view.ViewGroup
5	import androidx.recyclerview.widget.RecyclerView
6	import com.example.scrollablexml.databinding.HomeCarouselCardBi nding
7	import com.example.scrollablexml.model.ScrollableData
8	import com.example.scrollablexml.ui.home.viewholder.CarouselVie wHolder
9	
10	class CarouselAdapter(11 private val items: List<ScrollableData>, 12 private val onDetailClicked: (Int) -> Unit 13): RecyclerView.Adapter<CarouselViewHolder>() { 14 15 override fun onCreateViewHolder(parent: ViewGroup, viewType: Int): CarouselViewHolder { 16 val binding = HomeCarouselCardBinding.inflate(17 LayoutInflater.from(parent.context), parent, false 18) 19 return CarouselViewHolder(binding) 20 } 21 22 override fun onBindViewHolder(holder: CarouselViewHolder, position: Int) { 23 holder.bind(items[position], position, onDetailClicked) 24 } 25 26 override fun getItemCount() = items.size 27 }

Tabel 10. Source Code CarouselHeaderAdapter.Kt XML

1	package com.example.scrollablexml.ui.home.adapter
2	
3	import android.view.ViewGroup
4	import androidx.recyclerview.widget.RecyclerView
5	import com.example.scrollablexml.model.ScrollableData
6	import com.example.scrollablexml.ui.home.viewholder.CarouselHea derViewHolder
7	
8	class CarouselHeaderAdapter(9 private val items: List<ScrollableData>,

10	private val onDetailClick: (Int) -> Unit
11	): RecyclerView.Adapter<CarouselHeaderViewHolder>() {
12	override fun onCreateViewHolder(parent: ViewGroup,
13	viewType: Int)
14	= CarouselHeaderViewHolder.from(parent)
15	override fun onBindViewHolder(holder:
16	CarouselHeaderViewHolder, position: Int)
17	= holder.bind(items, onDetailClick)
18	override fun getItemCount() = 1
19	}

Tabel 11. Source Code ItemCardAdapter.Kt XML

1	package com.example.scrollablexml.ui.home.adapter
2	
3	import android.view.LayoutInflater
4	import android.view.ViewGroup
5	import androidx.recyclerview.widget.RecyclerView
6	import com.example.scrollablexml.databinding.
7	HomeItemCardBinding
8	import com.example.scrollablexml.model.ScrollableData
9	import
10	com.example.scrollablexml.ui.home.viewholder.ItemCardView
11	ViewHolder
12	
13	class ItemCardAdapter(
14	private val items: List<ScrollableData>,
15	private val onDetailClicked: (Int) -> Unit,
16	private val onSteamClicked: (String) -> Unit
17	): RecyclerView.Adapter<ItemCardViewHolder>() {
18	
19	override fun onCreateViewHolder(parent: ViewGroup,
20	viewType: Int): ItemCardViewHolder {
21	val binding = HomeItemCardBinding.inflate(
22	LayoutInflater.from(parent.context), parent,
23	false
24	)
25	return ItemCardViewHolder(binding)
26	}
27	
28	override fun onBindViewHolder(holder:
29	ItemCardViewHolder, position: Int) {
30	holder.bind(items[position], position,
31	onDetailClicked, onSteamClicked)
32	}

26	
27	override fun getItemCount() = items.size
28	}

Tabel 12. Source Code CarouselViewHolder.Kt XML

1	package com.example.scrollablexml.ui.home.viewholder
2	
3	import androidx.recyclerview.widget.RecyclerView
4	import com.example.scrollablexml.databinding. HomeCarouselCardBinding
5	import com.example.scrollablexml.model.ScrollableData
6	
7	class CarouselViewHolder(8 private val binding: HomeCarouselCardBinding 9): RecyclerView.ViewHolder(binding.root) { 10 fun bind(item: ScrollableData, index: Int, onDetailClicked: (Int) -> Unit) { 11 12 binding.ivCarousel.setImageResource(item.detailImageResourceId) 13 binding.root.setOnClickListener { onDetailClicked(index) } 14 } 15 }

Tabel 13. Source Code ItemCardViewHolder.Kt XML

1	package com.example.scrollablexml.ui.home.viewholder
2	
3	import androidx.recyclerview.widget.RecyclerView
4	import com.example.scrollablexml.databinding.HomeItemCardBinding
5	import com.example.scrollablexml.model.ScrollableData
6	
7	class ItemCardViewHolder(8 private val binding: HomeItemCardBinding 9): RecyclerView.ViewHolder(binding.root) { 10 11 fun bind(12 item: ScrollableData, 13 index: Int, 14 onDetailClicked: (Int) -> Unit, 15 onSteamClicked: (String) -> Unit 16) { 17 binding.apply {

18	ivItem.setImageResource(item.imageResourceId)
19	tvTitle.setText(item.titleResourceId)
20	tvSubtitle.setText(item.subtitleResourceId)
21	tvDesc.setText(item.descriptionResourceId)
22	btnDetail.setOnClickListener {
	onDetailClicked(index) }
23	btnSteam.setOnClickListener {
	onSteamClicked(item.steamUrl) }
24	}
25	}
26	}

Tabel 14. Source Code navigation_graph.xml

1	<?xml version="1.0" encoding="utf-8"?>
2	<navigation
	xmlns:android="http://schemas.android.com/apk/res/android"
3	xmlns:app="http://schemas.android.com/apk/res-auto"
4	android:id="@+id/navigation_graph"
5	app:startDestination="@+id/homeFragment">
6	
7	<fragment
8	android:id="@+id/homeFragment"
9	
	android:name="com.example.scrollablexml.ui.home.HomeFragment"
10	android:label="@string/home_label">
11	<action
12	
	android:id="@+id/action_homeFragment_to_settingsFragment"
13	app:destination="@id/settingsFragment"/>
14	<action
15	
	android:id="@+id/action_homeFragment_to_detailFragment"
16	app:destination="@id/detailFragment"/>
17	</fragment>
18	
19	<fragment
20	android:id="@+id/detailFragment"
21	
	android:name="com.example.scrollablexml.ui.detail.DetailFragment"
22	android:label="@string/detail_label">
23	<argument

24	android:name="itemIndex"
25	app:argType="integer"
26	android:defaultValue="0"/>
27	</fragment>
28	
29	<fragment
30	android:id="@+id/settingsFragment"
31	android:name="com.example.scrollablexml.ui.settings.SettingsFragment"
32	android:label="@string/setting_label"/>
33	
34	</navigation>

Tabel 15. Source Code activity_main.xml

1	<?xml version="1.0" encoding="utf-8"?>
2	<androidx.fragment.app.FragmentContainerView
	xmlns:android="http://schemas.android.com/apk/res/android"
3	xmlns:app="http://schemas.android.com/apk/res-auto"
4	xmlns:tools="http://schemas.android.com/tools"
5	android:id="@+id/main"
6	android:name="androidx.navigation.fragment.NavHostFragment"
7	android:layout_width="match_parent"
8	android:layout_height="match_parent"
9	app:defaultNavHost="true"
10	app:navGraph="@navigation/navigation_graph"
11	tools:layout="@layout/fragment_home" />

Tabel 16. Source Code fragment_detail.xml

1	<?xml version="1.0" encoding="utf-8"?>
2	<androidx.coordinatorlayout.widget.CoordinatorLayout
	xmlns:android="http://schemas.android.com/apk/res/android"
3	xmlns:app="http://schemas.android.com/apk/res-auto"
4	android:layout_width="match_parent"
5	android:layout_height="match_parent"
6	android:fitsSystemWindows="true">
7	
8	<com.google.android.material.appbar.AppBarLayout
9	android:layout_width="match_parent"
10	android:layout_height="wrap_content">
11	

```

12 <com.google.android.material.appbar.MaterialToolbar
13     android:id="@+id/toolbar"
14     android:layout_width="match_parent"
15     android:layout_height="wrap_content"
16     android:minHeight="?attr/actionBarSize"
17     app:titleCentered="true"
18     app:navigationIcon="@drawable/ic_arrow_back"
19     android:layout_marginTop="8dp"/>
20
21 </com.google.android.material.appbar.AppBarLayout>
22
23 <androidx.core.widget.NestedScrollView
24     android:layout_width="match_parent"
25     android:layout_height="match_parent"
26
27     app:layout_behavior="@string/appbar_scrolling_view_
28     behavior">
29
30     <LinearLayout
31         android:layout_width="match_parent"
32         android:layout_height="wrap_content"
33         android:padding="16dp"
34         android:orientation="vertical">
35
36         <ImageView
37             android:id="@+id/iv_detail"
38             android:layout_width="350dp"
39             android:layout_height="450dp"
40
41             android:contentDescription="@string/head_title"
42             android:layout_gravity="center"/>
43
44         <LinearLayout
45             android:layout_width="match_parent"
46             android:layout_height="wrap_content"
47             android:orientation="vertical"
48             android:padding="16dp">
49
50             <TextView
51                 android:id="@+id/tv_title"
52                 android:layout_width="wrap_content"
53                 android:layout_height="wrap_content"
54                 android:textAppearance="?attr/
55                 textAppearanceHeadlineMedium"/>

```

53	<TextView
54	android:id="@+id/tv_subtitle"
55	android:layout_width="wrap_content"
56	android:layout_height="wrap_content"
57	android:textAppearance="?attr/
	textAppearanceHeadlineSmall"/>
58	
59	<com.google.android.material.card.MaterialCardView
60	android:layout_width="match_parent"
61	android:layout_height="wrap_content"
62	android:layout_marginTop="8dp"
63	app:cardElevation="4dp"
64	app:strokeWidth="1dp"
65	app:strokeColor="?attr/colorOutline">
66	
67	<LinearLayout
68	android:layout_width="match_parent"
69	android:layout_height="wrap_content"
70	android:orientation="vertical"
71	android:padding="8dp">
72	
73	<TextView
74	android:id="@+id/tv_desc"
75	android:layout_width="wrap_content"
76	android:layout_height="wrap_content"
77	android:textAppearance="?attr/
	textAppearanceTitleLarge"/>
78	
79	<TextView
80	android:id="@+id/tv_detail"
81	android:layout_width="wrap_content"
82	android:layout_height="wrap_content"

83	android:textAppearance="?attr/textAppearanceBodyMedium"
84	android:justificationMode="inter_word"/>
85	
86	</LinearLayout>
87	
88	</com.google.android.material.card.MaterialCardView>
89	
90	</LinearLayout>
91	</LinearLayout>
92	</androidx.core.widget.NestedScrollView>
93	</androidx.coordinatorlayout.widget.CoordinatorLayout>

Tabel 17. Source Code fragment settings.xml

1	<?xml version="1.0" encoding="utf-8"?>
2	<androidx.coordinatorlayout.widget.CoordinatorLayout xmlns:android="http://schemas.android.com/apk/res/ android"
3	xmlns:app="http://schemas.android.com/apk/res-auto"
4	android:layout_width="match_parent"
5	android:layout_height="match_parent"
6	android:fitsSystemWindows="true">
7	
8	<com.google.android.material.appbar.AppBarLayout
9	android:layout_width="match_parent"
10	android:layout_height="wrap_content">
11	
12	<com.google.android.material.appbar.MaterialToolbar
13	android:id="@+id/toolbar"
14	android:layout_width="match_parent"
15	android:layout_height="wrap_content"
16	android:minHeight="?attr/actionBarSize"
17	app:title="@string/setting_button"
18	app:titleCentered="true"
19	app:navigationIcon="@drawable/ic_arrow_back"
20	android:layout_marginTop="8dp"/>
21	
22	</com.google.android.material.appbar.AppBarLayout>
23	
24	<androidx.core.widget.NestedScrollView
25	android:layout_width="match_parent"
26	android:layout_height="match_parent"


```

27         app:layout_behavior="@string/
appbar_scrolling_view_behavior">
28
29         <LinearLayout
30             android:layout_width="match_parent"
31             android:layout_height="wrap_content"
32             android:orientation="vertical"
33             android:padding="16dp">
34
35             <TextView
36                 android:layout_width="wrap_content"
37                 android:layout_height="wrap_content"
38                 android:text="@string/setting_header"
39                 android:textAppearance="?attr/
textAppearanceHeadlineSmall"
40                 android:layout_marginVertical="8dp"/>
41
42             <RadioGroup
43                 android:id="@+id/rg_locale"
44                 android:layout_width="wrap_content"
45                 android:layout_height="wrap_content"
46                 android:orientation="vertical">
47
48                 <RadioButton
49                     android:id="@+id/rb_english"
50                     android:layout_width="wrap_content"
51                     android:layout_height="wrap_content"
52                     android:text="@string/button_english"/>
53
54                 <RadioButton
55                     android:id="@+id/rb_indonesian"
56                     android:layout_width="wrap_content"
57                     android:layout_height="wrap_content"
58                     android:text="@string/button_indonesia"/>
59
60             </RadioGroup>
61
62         </LinearLayout>
63
64     </androidx.core.widget.NestedScrollView>
65
66 </androidx.coordinatorlayout.widget.CoordinatorLayout>

```

Tabel 18. Source Code fragment home.xml

1	<?xml version="1.0" encoding="utf-8"?>
2	<androidx.coordinatorlayout.widget.CoordinatorLayout xmlns:android="http://schemas.android.com/apk/res/ android"
3	android:layout_width="match_parent"
4	android:layout_height="match_parent"
5	android:fitsSystemWindows="true"
6	xmlns:app="http://schemas.android.com/apk/res-auto">
7	
8	<com.google.android.material.appbar.AppBarLayout
9	android:layout_width="match_parent"
10	android:layout_height="wrap_content">
11	
12	<com.google.android.material.appbar.MaterialToolbar
13	android:id="@+id/toolbar"
14	android:layout_width="match_parent"
15	android:layout_height="wrap_content"
16	android:minHeight="?attr/actionBarSize"
17	app:titleCentered="true"
18	
19	app:navigationIcon="@drawable/ic_exit_to_app"
20	android:layout_marginTop="8dp">
21	<TextView
22	android:id="@+id/tv_toolbar_title"
23	android:layout_width="wrap_content"
24	android:layout_height="wrap_content"
25	android:layout_gravity="center_vertical"
26	android:text="@string/head_title"
27	android:maxLines="2"
28	android:gravity="start"
29	android:textAppearance="?attr/ textAppearanceHeadlineSmall" />
30	
31	</com.google.android.material.appbar.MaterialToolbar>
32	
33	</com.google.android.material.appbar.AppBarLayout>
34	
35	<androidx.recyclerview.widget.RecyclerView
36	android:id="@+id/rv_main"
37	android:layout_width="match_parent"
38	android:layout_height="match_parent"
39	android:padding="8dp"

40	android:clipToPadding="false"
41	app:layout_behavior="@string/ appbar_scrolling_view_behavior"/>
42	
43	</androidx.coordinatorlayout.widget.CoordinatorLayout>

Tabel 19. Source Code menu homel.xml

1	<?xml version="1.0" encoding="utf-8"?>
2	<menu xmlns:android="http://schemas.android.com/apk/res/ android"
3	xmlns:app="http://schemas.android.com/apk/res-auto">
4	
5	<item
6	android:id="@+id/action_setting"
7	android:title="@string/setting_button"
8	android:icon="@drawable/ic_settings"
9	app:showAsAction="ifRoom"/>
10	
11	</menu>

Tabel 20. Source Code home carousel header.xml

1	<?xml version="1.0" encoding="utf-8"?>
2	<androidx.recyclerview.widget.RecyclerView xmlns:android="http://schemas.android.com/apk/res/ android"
3	android:id="@+id/rv_carousel"
4	android:layout_width="match_parent"
5	android:layout_height="wrap_content"
6	android:orientation="horizontal"/>

Tabel 21. Source Code home carousel card.xml

1	<?xml version="1.0" encoding="utf-8"?>
2	<com.google.android.material.card.MaterialCardView xmlns:android="http://schemas.android.com/apk/res/ android"
3	xmlns:app="http://schemas.android.com/apk/res-auto"
4	android:layout_width="match_parent"
5	android:layout_height="wrap_content"
6	android:layout_margin="8dp"
7	app:cardElevation="4dp"
8	app:strokeWidth="1dp"
9	app:strokeColor="?attr/colorOutline">
10	
11	<LinearLayout
12	android:layout_width="match_parent"
13	android:layout_height="wrap_content"

14	android:orientation="horizontal"
15	android:padding="16dp"
16	android:gravity="center">
17	
18	<com.google.android.material.imageview. ShapeableImageView
19	android:id="@+id/iv_carousel"
20	android:layout_width="400dp"
21	android:layout_height="200dp"
22	android:scaleType="centerCrop"
23	
	app:shapeAppearanceOverlay="@style/ShapeAppearance. Material3.Corner.Medium"/>
24	
25	</LinearLayout>
26	
27	</com.google.android.material.card.MaterialCardView>

Tabel 22. Source Code home item_card.xml

1	<?xml version="1.0" encoding="utf-8"?>
2	<com.google.android.material.card.MaterialCardView xmlns:android="http://schemas.android.com/apk/res/ android"
3	xmlns:app="http://schemas.android.com/apk/res-auto"
4	android:layout_width="match_parent"
5	android:layout_height="wrap_content"
6	android:layout_margin="8dp"
7	app:cardElevation="4dp"
8	app:strokeWidth="1dp"
9	app:strokeColor="?attr/colorOutline">
10	
11	<LinearLayout
12	android:layout_width="match_parent"
13	android:layout_height="wrap_content"
14	android:orientation="horizontal"
15	android:padding="16dp">
16	
17	<com.google.android.material.imageview. ShapeableImageView
18	android:id="@+id/iv_item"
19	android:layout_width="100dp"
20	android:layout_height="140dp"
21	android:scaleType="fitXY"
22	
	app:shapeAppearanceOverlay="@style/ShapeAppearance. Material3.Corner.Medium"/>

```

23
24         <LinearLayout
25             android:layout_width="0dp"
26             android:layout_height="wrap_content"
27             android:layout_weight="1"
28             android:layout_marginStart="16dp"
29             android:orientation="vertical">
30
31             <TextView
32                 android:id="@+id/tv_title"
33                 android:layout_width="wrap_content"
34                 android:layout_height="wrap_content"
35                 android:textAppearance="?attr/
textAppearanceHeadlineSmall"/>
36
37             <TextView
38                 android:id="@+id/tv_subtitle"
39                 android:layout_width="wrap_content"
40                 android:layout_height="wrap_content"
41
android:textAppearance="?attr/textAppearanceBodyLarge"/>
42
43             <TextView
44                 android:id="@+id/tv_desc"
45                 android:layout_width="wrap_content"
46                 android:layout_height="wrap_content"
47                 android:textAppearance="?attr/
textAppearanceBodyMedium"/>
48
49             <LinearLayout
50                 android:layout_width="match_parent"
51                 android:layout_height="wrap_content"
52                 android:layout_marginTop="24dp"
53                 android:orientation="horizontal"
54                 android:gravity="end">
55
56         <com.google.android.material.button.MaterialButton
57             android:id="@+id/btn_steam"
58             android:layout_width="wrap_content"
59             android:layout_height="wrap_content"
60             android:text="@string/steam_button"
61             android:maxLines="1"
62
android:textAppearance="?attr/textAppearanceBodySmall"
63             android:paddingHorizontal="16dp"/>

```

64	
65	<Space
66	android:layout_width="8dp"
67	
	android:layout_height="wrap_content"/>
68	
69	<com.google.android.material.button.MaterialButton
70	android:id="@+id/btn_detail"
71	android:layout_width="wrap_content"
72	android:layout_height="wrap_content"
73	android:text="@string/detail_button"
74	android:maxLines="1"
75	
	android:textAppearance="?attr/textAppearanceBodySmall"/>
76	</LinearLayout>
77	</LinearLayout>
78	</LinearLayout>
79	
80	</com.google.android.material.card.MaterialCardView>

Tabel 23. Source Code MainActivity.Kt Compose

1	package com.example.scrollablemodul3
2	
3	import android.os.Bundle
4	import androidx.appcompat.app.AppCompatActivity
5	import androidx.activity.compose.setContent
6	import androidx.activity.enableEdgeToEdge
7	import
	com.example.scrollablemodul3.ui.theme.ScrollableModul3Th
	eme
8	
9	class MainActivity : AppCompatActivity() {
10	override fun onCreate(savedInstanceState: Bundle?) {
11	super.onCreate(savedInstanceState)
12	enableEdgeToEdge()
13	setContent {
14	ScrollableModul3Theme {
15	ScrollableApp()
16	}
17	}
18	}
19	}

Tabel 24. Source Code ScrollableApp.Kt Compose

```

1 package com.example.scrollablemodul3
2
3 import android.app.Activity
4 import android.content.Intent
5 import androidx.appcompat.app.AppCompatActivity
6 import androidx.compose.runtime.Composable
7 import androidx.compose.runtime.LaunchedEffect
8 import androidx.compose.runtime.collectAsState
9 import androidx.compose.runtime.getValue
10 import androidx.compose.ui.Modifier
11 import androidx.compose.ui.platform.LocalContext
12 import androidx.core.net.toUri
13 import androidx.lifecycle.viewmodel.compose.viewModel
14 import androidx.navigation.NavHostController
15 import androidx.navigation.compose.NavHost
16 import androidx.navigation.compose.composable
17 import androidx.navigation.compose.rememberNavController
18 import com.example.scrollablemodul3.ui.screen.DetailScreen
19 import com.example.scrollablemodul3.ui.screen.HomeScreen
20 import com.example.scrollablemodul3.ui.ScrollableViewModel
21 import
    com.example.scrollablemodul3.ui.screen.SettingScreen
22
23 enum class ScrollableScreen { Home, Details, Settings }
24
25 @Composable
26 fun ScrollableApp(
27     navController: NavHostController =
    rememberNavController()
28 ) {
29     val defaultLocale =
    AppCompatActivity.getApplicationLocales().get(0)?.language
    ?: "en"
30     val viewModel: ScrollableViewModel = viewModel(
31         factory =
    ScrollableViewModel.Factory(defaultLocale)
32     )
33     val uiState by viewModel.uiState.collectAsState()
34     val context = LocalContext.current
35
36     NavHost(
37         navController = navController,
38         startDestination = ScrollableScreen.Home.name,
39     ) {
40         composable(route = ScrollableScreen.Home.name) {

```

```

41         val activity = context as Activity
42         HomeScreen(
43             uiState = uiState,
44             onDetailClick = { index ->
45
46                 navController.navigate("${ScrollableScreen.Details.name}/$
47                 index")
48                 viewModel.onDetailButtonClicked()
49             },
50             onIntentClick = { url ->
51
52                 context.startActivity(Intent(Intent.ACTION_VIEW,
53                 url.toUri()))
54                 viewModel.onIntentButtonClicked()
55             },
56             onSettingsClick = {
57                 navController.navigate(ScrollableScreen.Settings.name) },
58             onExit = { activity.finish() }
59         )
60         composable(route =
61         "${ScrollableScreen.Details.name}/{itemId}") {
62             backStackEntry ->
63                 val index =
64                 backStackEntry.arguments?.getString("itemId")?.toIntOrNull
65                 () ?: 0
66                 val item = uiState.list.getOrNull(index) ?:
67                 uiState.list.first()
68                 LaunchedEffect(index) {
69                     viewModel.updateCurrentItem(index)
70                 }
71                 DetailScreen(
72                     item = item,
73                     modifier = Modifier,
74                     onBackClick = { navController.navigateUp()
75                 }
76             )
77         }
78         composable(route = ScrollableScreen.Settings.name)
79         {
80             SettingScreen(
81                 modifier = Modifier,
82                 onBackClick = { navController.navigateUp()
83             },
84             onLocaleChange = { locale ->
85                 viewModel.updateLocale(locale) },

```


73	selectedLocale = uiState.selectedLocale
74	)
75	}
76	}
77	}

Tabel 25. Source Code ScrollableData.Kt Compose

1	package com.example.scrollablemodul3.model
2	
3	import androidx.annotation.DrawableRes
4	import androidx.annotation.StringRes
5	
6	data class ScrollableData(
7	val id: Int,
8	@StringRes val titleResourceId: Int,
9	@StringRes val subtitleResourceId: Int,
10	@StringRes val descriptionResourceId: Int,
11	@StringRes val detailResourceId: Int,
12	@DrawableRes val imageResourceId: Int,
13	@DrawableRes val detailImageResourceId: Int,
14	val steamUrl: String
15	)

Tabel 26. Source Code DataSource.Kt Compose

1	package com.example.scrollablemodul3.model
2	
3	import com.example.scrollablemodul3.R
4	
5	class DataSource {
6	fun loadItems(): List<ScrollableData> {
7	return listOf(
8	ScrollableData(1, R.string.item1,
	R.string.item1_sub, R.string.item1_desc,
	R.string.item1_detail, R.drawable.crosscode,
	R.drawable.crosscode_detail,
	"https://store.steampowered.com/app/368340/CrossCode/"),
9	ScrollableData(2, R.string.item2,
	R.string.item2_sub, R.string.item2_desc,
	R.string.item2_detail, R.drawable.hades2,
	R.drawable.hades2_detail,
	"https://store.steampowered.com/app/1145350/Hades_II/"),
10	ScrollableData(3, R.string.item3,
	R.string.item3_sub, R.string.item3_desc,
	R.string.item3_detail, R.drawable.nms,
	R.drawable.nms_detail,

11	"https://store.steampowered.com/app/275850/No_Mans_Sky/") , ScrollableData(4, R.string.item4, R.string.item4_sub, R.string.item4_desc, R.string.item4_detail, R.drawable.coe33, R.drawable.coe33_detail, "https://store.steampowered.com/app/1903340/Clair_Obscur _Expedition_33/"),
12	ScrollableData(5, R.string.item5, R.string.item5_sub, R.string.item5_desc, R.string.item5_detail, R.drawable.sekiro, R.drawable.sekiro_detail, "https://store.steampowered.com/app/814380/Sekiro_Shadow s_Die_Twice__GOTY_Edition/")
13	)
14	}
15	}

Tabel 27. Source Code ScrollableViewModel.Kt Compose

1	package com.example.scrollablemodul3.ui
2	
3	import androidx.appcompat.app.AppCompatActivityDelegate
4	import androidx.core.os.LocaleListCompat
5	import androidx.lifecycle.ViewModel
6	import androidx.lifecycle.ViewModelProvider
7	import androidx.lifecycle.viewmodel.initializer
8	import androidx.lifecycle.viewmodel.viewModelFactory
9	import com.example.scrollablemodul3.model.DataSource
10	import kotlinx.coroutines.flow.MutableStateFlow
11	import kotlinx.coroutines.flow.StateFlow
12	import kotlinx.coroutines.flow.asStateFlow
13	import timber.log.Timber
14	
15	class ScrollableViewModel(initialLocale: String) : ViewModel() {
16	private val _uiState = MutableStateFlow(ScrollableUiState(selectedLocale = initialLocale))
17	val uiState: StateFlow<ScrollableUiState> = _uiState.asStateFlow()
18	
19	init {
20	val items = DataSource().loadItems()
21	Timber.d("Items loaded: \${items.size}")
22	Timber.d("Items: \$items")
23	_uiState.value = ScrollableUiState(

```

24         list = items,
25         currentItemIndex = 0,
26         selectedLocale = initialLocale
27     )
28 }
29
30 fun updateCurrentItem(index: Int) {
31     val selectedItem =
32     _uiState.value.list.getOrNull(index)
33     Timber.d("Selected at index $index of itemId:
34     ${selectedItem?.id}")
35     Timber.d("Selected items: $selectedItem")
36
37     _uiState.value = _uiState.value.copy(
38         currentItemIndex = index
39     )
40 }
41
42 fun onDetailButtonClicked() {
43     Timber.d("Detail button clicked")
44 }
45
46 fun onIntentButtonClicked(){
47     Timber.d("Intent button clicked")
48 }
49
50 fun updateLocale(locale: String) {
51     _uiState.value = _uiState.value.copy(
52         selectedLocale = locale,
53     )
54     val appLocale =
55     LocaleListCompat.forLanguageTags(locale)
56     AppCompatActivity.setApplicationLocales(appLocale)
57 }
58
59 companion object {
60     fun Factory(initialLocale: String):
61     ViewModelProvider.Factory = viewModelFactory {
62         initializer {
63             ScrollableViewModel(initialLocale)
64         }
65     }
66 }
67
68 }

```

Tabel 28. Source Code ScrollableUiState.Kt Compose

1	package com.example.scrollablemodul3.ui
2	
3	import com.example.scrollablemodul3.model.ScrollableData
4	
5	data class ScrollableUiState(
6	val list: List<ScrollableData> = emptyList(),
7	val currentItemIndex: Int = 0,
8	val selectedLocale: String = "en"
9	)

Tabel 29. Source Code DetailScreen.Kt Compose

1	package com.example.scrollablemodul3.ui.screen
2	
3	import androidx.compose.foundation.BorderStroke
4	import androidx.compose.foundation.Image
5	import androidx.compose.foundation.layout.Column
6	import androidx.compose.foundation.layout.fillMaxWidth
7	import androidx.compose.foundation.layout.height
8	import androidx.compose.foundation.layout.padding
9	import androidx.compose.foundation.rememberScrollState
10	import
	androidx.compose.foundation.shape.RoundedCornerShape
11	import androidx.compose.foundation.verticalScroll
12	import androidx.compose.material.icons.Icons
13	import
	androidx.compose.material.icons.automirrored.filled.
	ArrowBack
14	import androidx.compose.material3.Card
15	import androidx.compose.material3.CardDefaults
16	import androidx.compose.material3.CenterAlignedTopAppBar
17	import
	androidx.compose.material3.ExperimentalMaterial3Api
18	import androidx.compose.material3.Icon
19	import androidx.compose.material3.IconButton
20	import androidx.compose.material3.MaterialTheme
21	import androidx.compose.material3.Scaffold
22	import androidx.compose.material3.Text
23	import androidx.compose.runtime.Composable
24	import androidx.compose.ui.Modifier
25	import androidx.compose.ui.res.painterResource
26	import androidx.compose.ui.res.stringResource
27	import androidx.compose.ui.text.style.TextAlign
28	import androidx.compose.ui.tooling.preview.Preview
29	import androidx.compose.ui.unit.dp
30	import com.example.scrollablemodul3.R

```

31 import com.example.scrollablemodul3.model.ScrollableData
32
33 @OptIn(ExperimentalMaterial3Api::class)
34 @Composable
35 fun DetailScreen(
36     item: ScrollableData,
37     onBackPressed: () -> Unit,
38     modifier: Modifier = Modifier
39 ){
40     Scaffold(
41         topBar = {
42             CenterAlignedTopAppBar(
43                 title = {
44                     Text(text = stringResource(id =
item.titleResourceId))
45                 },
46                 navigationIcon = {
47                     IconButton(onClick = onBackPressed) {
48                         Icon(
49                             imageVector =
Icons.AutoMirrored.Filled.ArrowBack,
50                             contentDescription =
stringResource(R.string.back_button)
51                         )
52                     }
53                 }
54             )
55         }
56     ) { innerPadding ->
57         Column(
58             modifier = modifier
59                 .padding(innerPadding)
60                 .verticalScroll(rememberScrollState())
61         ) {
62             Image(
63                 painter = painterResource(id =
item.imageResourceId),
64                 contentDescription = stringResource(id =
item.titleResourceId),
65                 modifier = Modifier
66                     .fillMaxWidth()
67                     .height(450.dp)
68             )
69
70             Column(modifier = Modifier.padding(16.dp)) {
71                 Text(

```

```

72         text = stringResource(id =
item.titleResourceId),
73         style =
MaterialTheme.typography.headlineMedium
74     )
75     Text(
76         text = stringResource(id =
item.subtitleResourceId),
77         style =
MaterialTheme.typography.headlineSmall
78     )
79     Card(
80         modifier = Modifier
81             .fillMaxWidth()
82             .padding(top = 8.dp),
83         elevation =
CardDefaults.cardElevation(defaultElevation = 4.dp),
84         shape = RoundedCornerShape(8.dp),
85         border = BorderStroke(1.dp,
MaterialTheme.colorScheme.outline)
86     ) {
87         Column(modifier =
Modifier.padding(8.dp)) {
88             Text(
89                 text = stringResource(id =
item.descriptionResourceId),
90                 style =
MaterialTheme.typography.titleLarge
91             )
92             Text(
93                 text = stringResource(id =
item.detailResourceId),
94                 style =
MaterialTheme.typography.bodyMedium,
95                 textAlign =
TextAlign.Justify
96             )
97         }
98     }
99 }
100 }
101 }
102 }
103
104 @Preview
105 @Composable

```

106	fun DetailScreenPreview() {
107	DetailScreen(
108	item = ScrollableData(
109	id = 3,
110	titleResourceId = R.string.item3,
111	subtitleResourceId = R.string.item3_sub,
112	descriptionResourceId = R.string.item3_desc,
113	detailResourceId = R.string.item3_detail,
114	imageResourceId = R.drawable.nms,
115	detailImageResourceId =
	R.drawable.nms_detail,
116	steamUrl =
	"https://store.steampowered.com/app/1091500/Sekiro_Shadows_Die_Twice/"
117	),
118	modifier = Modifier,
119	onClick = {}
120	)
121	}

Tabel 30. Source Code SettingScreen.Kt Compose

1	package com.example.scrollablemodul3.ui.screen
2	
3	import androidx.compose.foundation.layout.Column
4	import androidx.compose.foundation.layout.Row
5	import androidx.compose.foundation.layout.fillMaxWidth
6	import androidx.compose.foundation.layout.padding
7	import androidx.compose.foundation.rememberScrollState
8	import androidx.compose.foundation.verticalScroll
9	import androidx.compose.material.icons.Icons
10	import androidx.compose.material.icons.automirrored.filled.ArrowBack
11	import androidx.compose.material3.CenterAlignedTopAppBar
12	import
	androidx.compose.material3.ExperimentalMaterial3Api
13	import androidx.compose.material3.Icon
14	import androidx.compose.material3.IconButton
15	import androidx.compose.material3.MaterialTheme
16	import androidx.compose.material3.RadioButton
17	import androidx.compose.material3.Scaffold
18	import androidx.compose.material3.Text
19	import androidx.compose.runtime.Composable
20	import androidx.compose.ui.Alignment
21	import androidx.compose.ui.Modifier
22	import androidx.compose.ui.res.stringResource
23	import androidx.compose.ui.tooling.preview.Preview

```

24 import androidx.compose.ui.unit.dp
25 import com.example.scrollablemodul3.R
26
27 @OptIn(ExperimentalMaterial3Api::class)
28 @Composable
29 fun SettingScreen(
30     modifier: Modifier = Modifier,
31     onBackClick: () -> Unit,
32     onLocaleChange: (String) -> Unit,
33     selectedLocale: String
34 ){
35     Scaffold(
36         topBar = {
37             CenterAlignedTopAppBar(
38                 title = {
39                     Text(text =
stringResource(R.string.setting_button))
40                 },
41                 navigationIcon = {
42                     IconButton(onClick = onBackClick) {
43                         Icon(
44                             imageVector =
Icons.AutoMirrored.Filled.ArrowBack,
45                             contentDescription =
stringResource(R.string.back_button)
46                         )
47                     }
48                 }
49             )
50         }
51     ) { innerPadding ->
52         Column(
53             modifier = modifier
54                 .padding(innerPadding)
55                 .padding(16.dp)
56                 .fillMaxWidth()
57                 .verticalScroll(rememberScrollState())
58         ) {
59             Text(
60                 text =
stringResource(R.string.setting_header),
61                 style =
MaterialTheme.typography.headlineSmall,
62                 modifier = Modifier.padding(vertical =
8.dp)
63             )

```


64	
65	Row(
66	verticalAlignment =
	Alignment.CenterVertically,
67	modifier = Modifier.fillMaxWidth()
68	) {
69	RadioButton(
70	selected = selectedLocale == "en",
71	onClick = { onLocaleChange("en") }
72	)
73	Text(text =
	stringResource(R.string.english_button))
74	}
75	Row(
76	verticalAlignment =
	Alignment.CenterVertically,
77	modifier = Modifier.fillMaxWidth()
78	) {
79	RadioButton(
80	selected = selectedLocale == "in",
81	onClick = { onLocaleChange("in") }
82	)
83	Text(text =
	stringResource(R.string.indonesian_button))
84	}
85	}
86	}
87	}
88	
89	@Preview
90	@Composable
91	fun SettingScreenLayout() {
92	SettingScreen(
93	modifier = Modifier,
94	onBackClick = {},
95	onLocaleChange = {},
96	selectedLocale = "en"
97	)
98	}

Tabel 31. Source Code HomeScreen.Kt Compose

1	package com.example.scrollablemodul3.ui.screen
2	
3	import androidx.compose.foundation.BorderStroke
4	import androidx.compose.foundation.Image

5	import androidx.compose.foundation.gestures.snapping. rememberSnapFlingBehavior
6	import androidx.compose.foundation.layout.Arrangement
7	import androidx.compose.foundation.layout.Column
8	import androidx.compose.foundation.layout.PaddingValues
9	import androidx.compose.foundation.layout.Row
10	import androidx.compose.foundation.layout.Spacer
11	import androidx.compose.foundation.layout.fillMaxWidth
12	import androidx.compose.foundation.layout.height
13	import androidx.compose.foundation.layout.padding
14	import androidx.compose.foundation.layout.width
15	import androidx.compose.foundation.lazy.LazyColumn
16	import androidx.compose.foundation.lazy.LazyRow
17	import androidx.compose.foundation.lazy.itemsIndexed
18	import androidx.compose.foundation.lazy.rememberLazyListState
19	import androidx.compose.foundation.shape.RoundedCornerShape
20	import androidx.compose.material.icons.Icons
21	import androidx.compose.material.icons.automirrored.filled. ExitToApp
22	import androidx.compose.material.icons.filled.Settings
23	import androidx.compose.material3.Button
24	import androidx.compose.material3.Card
25	import androidx.compose.material3.CardDefaults
26	import androidx.compose.material3.ExperimentalMaterial3Api
27	import androidx.compose.material3.Icon
28	import androidx.compose.material3.IconButton
29	import androidx.compose.material3.MaterialTheme
30	import androidx.compose.material3.Scaffold
31	import androidx.compose.material3.Text
32	import androidx.compose.material3.TopAppBar
33	import androidx.compose.runtime.Composable
34	import androidx.compose.ui.Modifier
35	import androidx.compose.ui.draw.clip
36	import androidx.compose.ui.layout.ContentScale
37	import androidx.compose.ui.res.painterResource
38	import androidx.compose.ui.res.stringResource
39	import androidx.compose.ui.tooling.preview.Preview
40	import androidx.compose.ui.unit.dp
41	import com.example.scrollablemodul3.R
42	import com.example.scrollablemodul3.model.ScrollableData
43	import com.example.scrollablemodul3.ui.ScrollableUiState
44	

```

45 @OptIn(ExperimentalMaterial3Api::class)
46 @Composable
47 fun HomeAppBar(
48     onSettingClick: () -> Unit,
49     onExit: () -> Unit
50 ) {
51     TopAppBar(
52         title = {
53             Text(stringResource(R.string.head_title))
54         },
55         navigationIcon = {
56             IconButton(onClick = onExit) {
57                 Icon(
58                     imageVector =
59 Icons.AutoMirrored.Filled.ExitToApp,
60                     contentDescription =
61 stringResource(R.string.back_button)
62                 )
63             }
64         },
65         actions = {
66             IconButton(onClick = onSettingClick) {
67                 Icon(
68                     imageVector =
69 Icons.Default.Settings,
70                     contentDescription =
71 stringResource(R.string.setting_button)
72                 )
73             }
74         }
75     )
76 }
77 @Composable
78 fun HomeScreen(
79     uiState: ScrollableUiState,
80     onDetailClick: (Int) -> Unit,
81     onIntentClick: (String) -> Unit,
82     onSettingsClick: () -> Unit,
83     onExit: () -> Unit,
84     modifier: Modifier = Modifier
85 ){
86     Scaffold(
87         topBar = {
88             HomeAppBar(
89                 onSettingClick = onSettingsClick,
90                 onExit = onExit

```

```

87         )
88     }
89     ) { innerPadding ->
90         LazyColumn(modifier = modifier
91             .padding(innerPadding)
92             .padding(start = 8.dp)
93         ) {
94             item {
95                 ItemCarousel(
96                     items = uiState.list,
97                     onDetailClick = { index ->
onDetailClick(index) }
98                 )
99             }
100             itemsIndexed(uiState.list) { index, item ->
101                 ItemCard(
102                     item = item,
103                     onDetailClick = {
onDetailClick(index) },
104                     onIntentClick = {
onIntentClick(item.steamUrl) }
105                 )
106             }
107         }
108     }
109 }
110
111 @Composable
112 fun ItemCard(
113     item: ScrollableData,
114     onDetailClick: () -> Unit,
115     onIntentClick: () -> Unit,
116     modifier: Modifier = Modifier
117 ) {
118     Card(
119         modifier = modifier
120             .padding(8.dp)
121             .fillMaxWidth(),
122         shape = MaterialTheme.shapes.medium,
123         elevation =
CardDefaults.cardElevation(defaultElevation = 4.dp),
124         border = BorderStroke(1.dp,
MaterialTheme.colorScheme.outline)
125     ) {
126         Row(modifier = Modifier.padding(16.dp)) {
127             Image(

```

```

128         painter = painterResource(id =
item.imageResourceId),
129         contentDescription = stringResource(id =
item.titleResourceId),
130         contentScale = ContentScale.FillBounds,
131         modifier = Modifier
132             .width(100.dp)
133             .height(140.dp)
134             .clip(RoundedCornerShape(16.dp))
135     )
136     Column(
137         modifier = Modifier
138             .padding(start = 16.dp)
139             .weight(1f)
140     ) {
141         Text(
142             text = stringResource(id =
item.titleResourceId),
143             style =
MaterialTheme.typography.headlineSmall
144         )
145         Text(
146             text = stringResource(id =
item.subtitleResourceId),
147             style =
MaterialTheme.typography.bodyLarge
148         )
149         Text(
150             text = stringResource(id =
item.descriptionResourceId),
151             style =
MaterialTheme.typography.bodyMedium
152         )
153     }
154     Row(
155         modifier = Modifier
156             .padding(top = 24.dp)
157             .fillMaxWidth(),
158         horizontalArrangement =
Arrangement.End,
159     ) {
160         Button(
161             onClick = onIntentClick,
162             contentPadding =
PaddingValues(horizontal = 16.dp)
163         ) {

```

```

164         Text(
165             text =
stringResource(R.string.steam_button),
166             style =
MaterialTheme.typography.bodySmall
167         )
168     }
169     Spacer(modifier =
Modifier.padding(8.dp))
170     Button(
171         onClick = onDetailClick,
172     ) {
173         Text(
174             text =
stringResource(R.string.detail_button),
175             style =
MaterialTheme.typography.bodySmall
176         )
177     }
178 }
179 }
180 }
181 }
182 }
183
184 @Composable
185 fun CarouselCard(
186     item: ScrollableData,
187     onDetailClick: () -> Unit,
188     modifier: Modifier = Modifier
189 ){
190     Card(
191         onClick = onDetailClick,
192         modifier = modifier.padding(8.dp),
193         shape = MaterialTheme.shapes.medium,
194         elevation =
CardDefaults.cardElevation(defaultElevation = 4.dp),
195         border = BorderStroke(1.dp,
MaterialTheme.colorScheme.outline),
196         colors = CardDefaults.cardColors(
197             containerColor =
MaterialTheme.colorScheme.surfaceContainer
198         )
199     ) {
200         Row(
201             modifier = Modifier

```

```

202         .padding(16.dp)
203         .fillMaxWidth(),
204         horizontalArrangement =
Arrangement.SpaceEvenly
205     ) {
206         Image(
207             painter = painterResource(id =
item.detailImageResourceId),
208             contentDescription = stringResource(id =
item.titleResourceId),
209             contentScale = ContentScale.Crop,
210             modifier = Modifier
211                 .width(400.dp)
212                 .height(200.dp)
213                 .clip(RoundedCornerShape(16.dp))
214         )
215     }
216 }
217
218 }
219
220 @Composable
221 fun ItemCarousel(
222     items: List<ScrollableData>,
223     onDetailClick: (Int) -> Unit,
224 ) {
225     val listState = rememberLazyListState()
226     LazyRow(
227         state = listState,
228         flingBehavior =
rememberSnapFlingBehavior(listState)
229     ) {
230         itemsIndexed(items) { index, item ->
231             CarouselCard(
232                 item = item,
233                 onDetailClick = { onDetailClick(index)
},
234                 modifier = Modifier.fillParentMaxWidth()
235             )
236         }
237     }
238 }
239
240 @Preview
241 @Composable
242 fun HomeScreenPreview() {

```

```

243     HomeScreen(
244         uiState = ScrollableUiState(
245             list = listOf(
246                 ScrollableData(
247                     id = 1,
248                     titleResourceId = R.string.item1,
249                     subtitleResourceId =
250 R.string.item1_sub,
251                     descriptionResourceId =
252 R.string.item1_desc,
253                     detailResourceId =
254 R.string.item1_detail,
255                     imageResourceId =
256 R.drawable.crosscode,
257                     detailImageResourceId =
258 R.drawable.crosscode_detail,
259                     steamUrl =
260 "https://store.steampowered.com/app/368340/CrossCode/"
261                 ),
262                 ScrollableData(
263                     id = 2,
264                     titleResourceId = R.string.item2,
265                     subtitleResourceId =
266 R.string.item2_sub,
267                     descriptionResourceId =
268 R.string.item2_desc,
269                     detailResourceId =
270 R.string.item2_detail,
271                     imageResourceId = R.drawable.hades2,
272                     detailImageResourceId =
273 R.drawable.hades2_detail,
274                     steamUrl =
275 "https://store.steampowered.com/app/368340/CrossCode/"
276                 ),
277                 ScrollableData(
278                     id = 3,
279                     titleResourceId = R.string.item3,
280                     subtitleResourceId =
281 R.string.item3_sub,
282                     descriptionResourceId =
283 R.string.item3_desc,
284                     detailResourceId =
285 R.string.item3_detail,
286                     imageResourceId = R.drawable.nms,
287                     detailImageResourceId =
288 R.drawable.nms_detail,

```

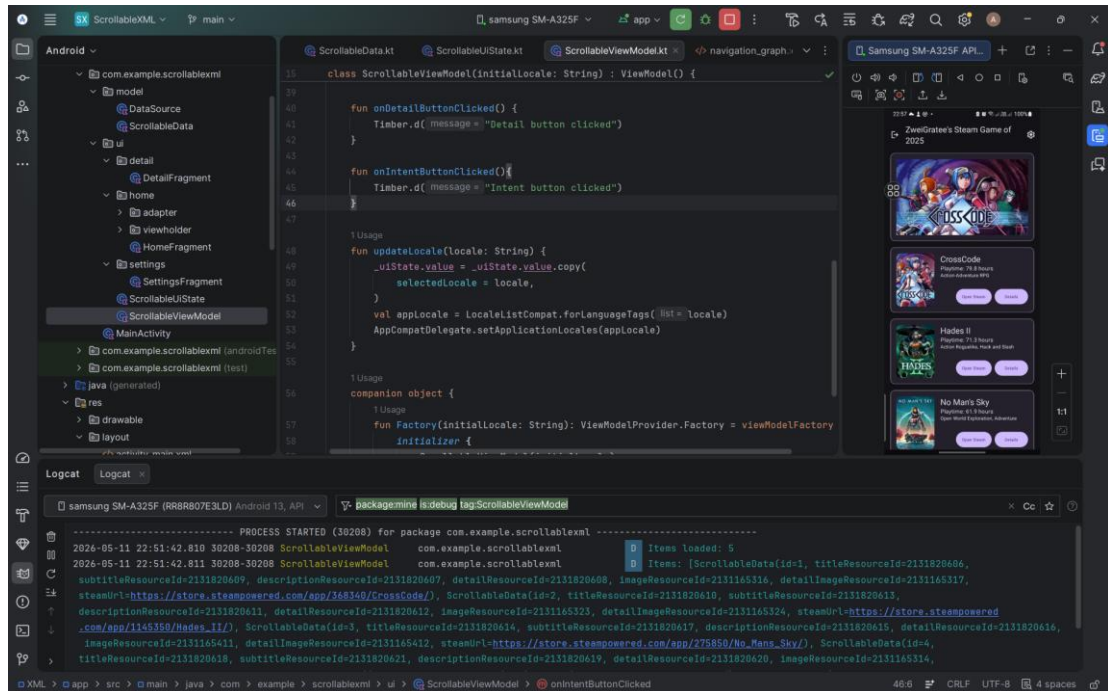


```

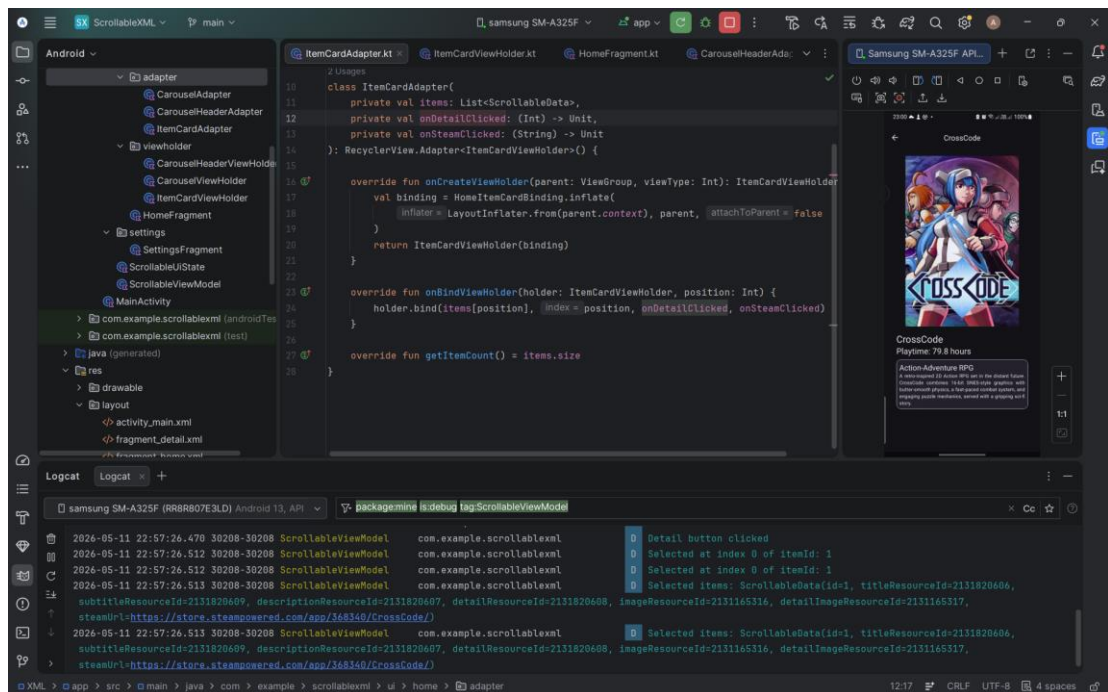
274         steamUrl =
275         "https://store.steampowered.com/app/368340/CrossCode/"
276         ),
277         ScrollableData(
278             id = 4,
279             titleResourceId = R.string.item4,
280             subtitleResourceId =
281             R.string.item4_sub,
282             descriptionResourceId =
283             R.string.item4_desc,
284             detailResourceId =
285             R.string.item4_detail,
286             imageResourceId = R.drawable.coe33,
287             detailImageResourceId =
288             R.drawable.coe33_detail,
289             steamUrl =
290             "https://store.steampowered.com/app/368340/CrossCode/"
291             ),
292             ScrollableData(
293                 id = 5,
294                 titleResourceId = R.string.item5,
295                 subtitleResourceId =
296                 R.string.item5_sub,
297                 descriptionResourceId =
298                 R.string.item5_desc,
299                 detailResourceId =
300                 R.string.item5_detail,
301                 imageResourceId = R.drawable.sekiro,
302                 detailImageResourceId =
303                 R.drawable.sekiro_detail,
304                 steamUrl =
305                 "https://store.steampowered.com/app/368340/CrossCode/"
306                 ),
307             ),
308             onDetailClick = {},
309             onIntentClick = {},
310             onSettingsClick = {},
311             onExit = {}
312         )
313     }

```

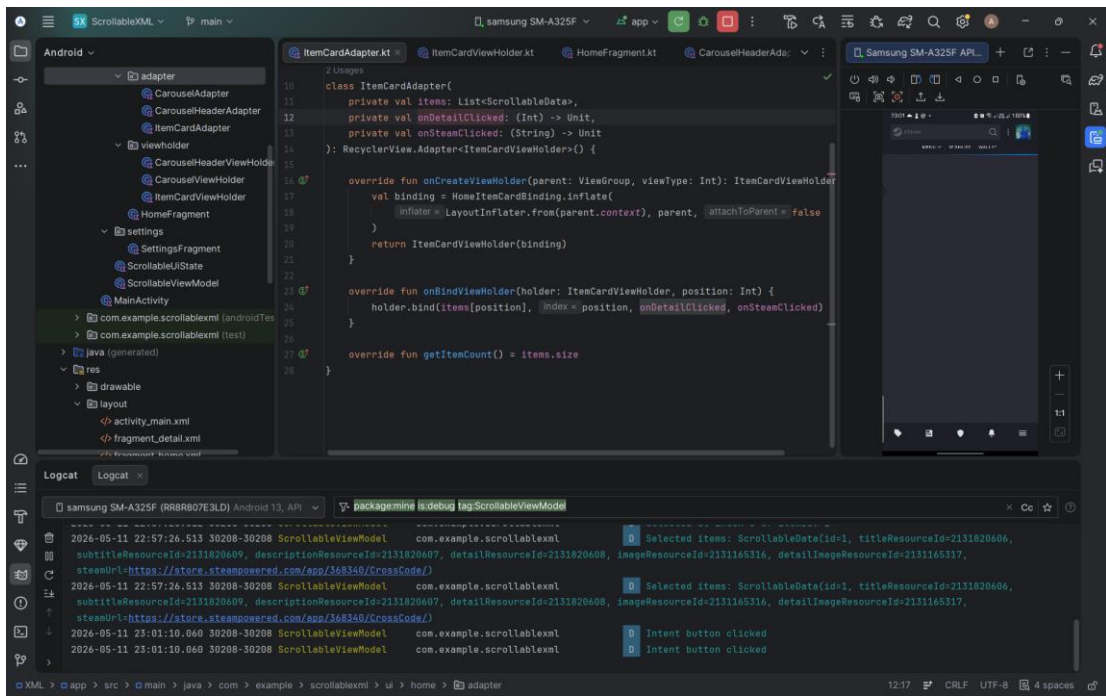
B. Output Program



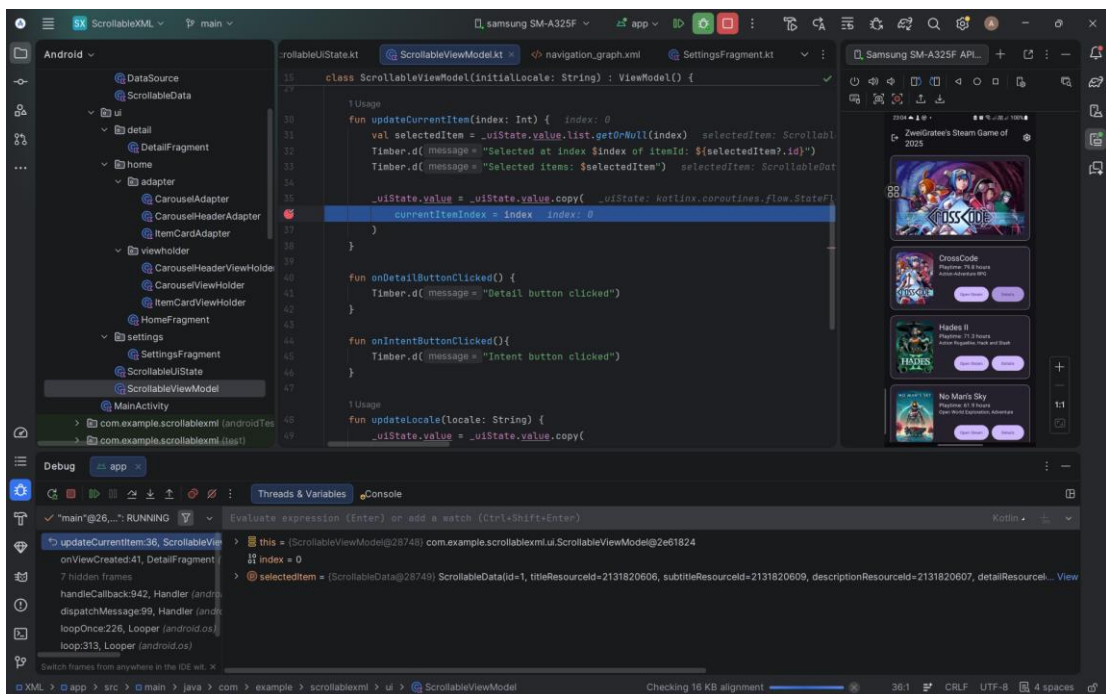
Gambar 1. Screenshot Hasil Log Loading Data XML



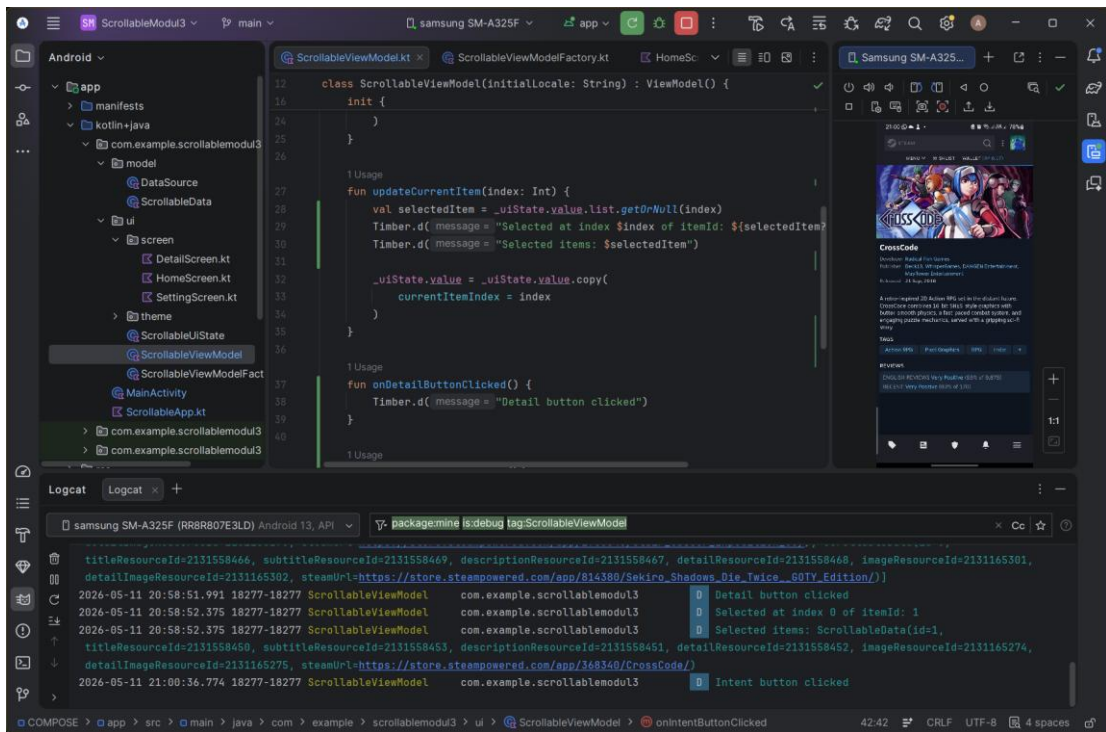
Gambar 2. Screenshot Hasil Log Tombol Detail dan List Data XML



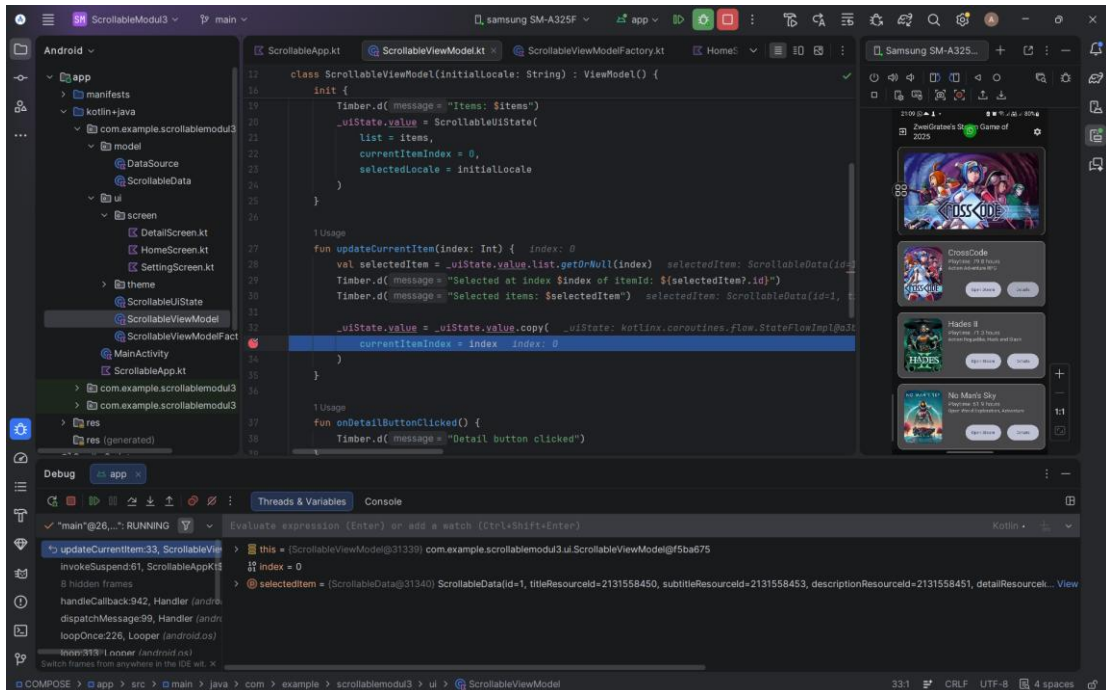
Gambar 3. Screenshot Hasil Log Tombol Intent XML



Gambar 4. Screenshot Hasil Debugging XML



Gambar 7. Screenshot Hasil Log Tombol Intent Compose



Gambar 8. Screenshot Hasil Debugging Compose

C. Pembahasan

ScrollableData.Kt & DataSource.Kt

ScrollableData.Kt merupakan data class yang berfungsi untuk menyimpan jenis-jenis data yang akan dimuat ke dalam aplikasi dengan data berupa id bertipe data Integer, data title, subtitle, description, dan detail yang merupakan String Resource, data image dan detail image yang merupakan Drawable Resource, dan data URL steam bertipe data String.

Data-data yang didefinisikan pada ScrollableData.Kt kemudian dimuat dengan method “loadItems()” yang akan memuat data-data hardcoded ke dalam aplikasi. Data yang dimuat disimpan dalam variable bertipe data List dan terdapat 5 data hardcoded yang akan dimuat ke dalam aplikasi yang nantinya akan dikelola oleh ViewModel.

ScrollableUiState.Kt & ScrollableViewModel.Kt

ScrollableUiState.Kt merupakan data class StateFlow yang menyimpan dan menginisialisasikan data berupa state atau kondisi UI yang terdapat pada aplikasi. State yang disimpan pada data class tersebut adalah “list” yang menyimpan data terkait list item yang akan ditampilkan pada Home Page, “currentItemIndex” yang menyimpan indeks list item yang berfungsi sebagai helper untuk navigasi ke Detail Page, dan “selectedLocale” yang menyimpan konfigurasi bahasa Aplikasi.

ScrollableViewModel.Kt merupakan class yang berfungsi untuk mengelola logika UI aplikasi dengan menerapkan arsitektur Model-View-ViewModel yang memisahkan data, logika bisnis, dan logika UI. Pada inisialisasi ViewModel, data-data aplikasi akan diinisialisasi, indeks list item diinisialisasi, dan locale aplikasi akan diambil kemudian data tersebut disimpan pada ScrollabelUiState.

ScrollableViewModel.Kt memiliki parameter “initialLocale” bertipe data String yang memuat locale Aplikasi yang nantinya akan diinisialisasi melalui ViewModelFactory. Selain itu, ada dua method khusus untuk tombol detail dan intent yang mengirimkan log ke Logcat serta terdapat log yang dikirimkan pada saat inisialisasi aplikasi dan log data yang dipilih ke Logcat.

Method “updateCurrentItem()” berfungsi untuk memperbarui indeks list item yang dipilih pada ScrollabelUiState.Kt yang nantinya akan digunakan untuk melakukan navigasi ke Detail Page sesuai dengan indeks item yang dipilih. Method “updateLocale()” berfungsi untuk mengatur locale atau konfigurasi bahasa aplikasi dengan memperbarui locale pada ScrollableUiState.Kt kemudian mengubah konfigurasi bahasa sesuai parameter.

MainActivity.Kt, activity_main.xml & navigation_graph.xml

MainActivity.Kt berfungsi sebagai Activity/UI untuk Aplikasi android yang akan memuat UI yang diambil dari layout file XML. Melalui ViewBinding, terdapat inisialisasi variabel “binding” yang berfungsi sebagai view binder ke layout activity_main.xml yang akan dihubungkan saat Activity menjalankan method “onCreate()” saat aplikasi pertama kali dibuka. Styling aplikasi akan extend ke navigation bar dan status bar, tetapi komponen UI tidak akan membelakangi kedua bar tersebut. Selain itu, terdapat inisialisasi ViewModel dan ViewModelFactory yang memuat locale aplikasi yang nantinya akan diterapkan pada masing-masing fragment.

activity_main.xml berfungsi sebagai layar utama yang akan dimuat saat method “onCreate()” dijalankan oleh Activity. Layar utama ini hanya berfungsi sebagai navigator yang akan digunakan oleh komponen navigasi pada file navigation_graph.xml yang memuat tiga Fragment yang telah dibuat dan dihubungkan pada navigation_graph.xml dengan layout default berupa Fragment Homepage.

navigation_graph.xml berfungsi untuk mengatur navigasi antar layar atau UI dengan memuat Fragment suatu page melalui method “onCreateView” dan di-“hancurkan” melalui method “onDestroyView” pada masing-masing fragment. Pada komponen navigasi, terdapat tiga Fragment yang dihubungkan, yaitu fragment_home.xml, fragment_detail.xml, dan fragment_settings.xml. Pada Fragment Home, terdapat dua action yang menghubungkan fragment tersebut ke Fragment Detail dan Fragment Setting, dimana action tersebut digunakan pada file HomeFragment.Kt.

fragment_settings.xml & SettingsFragment.Kt XML

fragment_settings.xml merupakan layout yang digunakan oleh Fragment Setting Page yang terdiri dari “CoordinatorLayout” yang merupakan layout yang berfungsi untuk memuat Topbar dan layar UI dibawahnya. Topbar didefinisikan dengan ViewGroup “AppBarLayout” yang memuat layout Topbar dan “MaterialToolbar” dengan padding atas 8dp yang memuat tampilan Topbar seperti ikon navigasi dengan desain Material 3 dan teks title.

Tampilan layar pada Fragment Settings diatur dengan “NestedScrollView” yang merupakan “ScrollView” modern yang diterapkan di dalam “CoordinatorLayout” sebagai layar scrollable. Di dalamnya, terdapat “LinearLayout” dengan orientasi vertikal dan padding keseluruhan 16dp yang memuat View “TextView” yang memuat teks dengan gaya Material 3 Headline dan margin vertikal 8dp. Kemudian, terdapat ViewGroup “RadioGroup” yang berfungsi untuk memuat kumpulan View “RadioButton” dengan orientasi vertikal.

SettingsFragment.Kt merupakan Fragment yang berupa “mini activity” yang lebih ringan dibandingkan penggunaan activity langsung sebagai Settings Page yang menampilkan UI berupa fragment_settings.xml dengan lifecycle tersendiri. Pada siklus “onCreateView()”, Fragment akan membangun UI dari file fragment_setting.xml. Setelah Fragment dibangun atau pada siklus “onViewCreated()”, Fragment menghubungkan listener ke tombol-tombol untuk navigasi back dan konfigurasi locale

yang terdapat pada UI. Jika Fragment ditutup atau pada siklus “onViewDestroy()”, Fragment akan melepas layout XML yang telah dibangun dengan memindahkan pointer ke nilai null.

fragment_detail & DetailFragment.Kt XML

fragment_detail.xml merupakan layout yang digunakan oleh Fragment Detail Page yang terdiri dari “CoordinatorLayout” berfungsi untuk memuat Topbar dengan “AppBarLayout” dan layar UI dibawahnya dengan “NestedScrollView”. Topbar didefinisikan dengan ViewGroup “AppBarLayout” yang memuat layout Topbar dan View “MaterialToolbar” dengan padding atas 8dp yang memuat tampilan Topbar seperti ikon navigasi dengan desain Material 3 dan teks title.

Tampilan layar pada Fragment Detail diatur dengan “NestedScrollView” yang berfungsi sebagai layar scrollable. Di dalamnya, terdapat “LinearLayout” vertikal dengan padding keseluruhan 16dp yang memuat “ImageView” dengan tinggi 350dp dan lebar 450dp dan orientasi gambar tengah. Di bawahnya, terdapat “LinearLayout” vertikal dengan padding keseluruhan 16dp yang memuat “TextView” title dan subtitle dengan gaya Material 3 Headline.

Dibawah “TextView title dan subtitle, terdapat ViewGroup “MaterialCardView” yang memuat layout Card atau kartu yang menampilkan suatu tempat dengan margin atas 8dp, elevasi kartu 4dp, ketebalan outline 1dp dengan warna Material 3 Outline. Di dalamnya terdapat “LinearLayout” vertikal dengan padding keseluruhan 8dp yang memuat “TextView” deskripsi yang memakai gaya Material 3 Title dan detail yang memakai gaya Material 3 Body.

DetailFragment.Kt merupakan Fragment yang berfungsi sebagai Detail Page yang menampilkan UI dengan lifecycle tersendiri. Pada siklus “onCreateView()”, Fragment akan membangun UI dari file fragment_detail.xml. Setelah Fragment dibangun atau pada siklus “onViewCreated()”, Fragment menghubungkan data-data yang dimuat ke dalam komponen teks dan gambar serta navigasi back. Jika Fragment

ditutup atau pada siklus “onViewDestroy()”, Fragment akan melepas layout XML yang telah dibangun dengan memindahkan pointer ke nilai null.

fragment_home.xml, menu_home.xml, & HomeFragment.Kt XML

fragment_home.xml merupakan layout yang digunakan oleh Fragment HomePage yang terdiri dari “CoordinatorLayout” yang memuat Topbar dengan ViewGroup “AppBarLayout” yang terdiri dari “MaterialToolbar” dengan padding atas 8dp yang menampilkan ikon navigasi exit serta menu_home.xml yang menampilkan ikon navigasi setting. Di dalam “MaterialToolbar”, terdapat view “TextView” sebagai title dengan orientasi tengah vertikal dan teks align left dan 2 baris teks dengan gaya Material 3 Headline. Dibawahnya terdapat ViewGroup “RecyclerView” yang menampilkan list data banyak secara dinamis efisien dengan mendaur ulang elemen View.

HomeFragment.Kt merupakan Fragment yang berfungsi sebagai Homepage. Pada siklus “onCreateView()”, Fragment akan membangun UI dari file fragment_home.xml. Pada siklus “onViewCreated()”, Fragment menghubungkan listener ke tombol-tombol navigasi pada item list dan Carousel, intent ke Steam Page, dan exit yang terdapat pada UI serta menghubungkan Adapter dan data ke UI. Pada tombol, terdapat panggilan method ke ViewModel yang mencetak log ke Logcat. Jika Fragment ditutup atau pada siklus “onViewDestroy()”, Fragment akan melepas layout XML yang telah dibangun dengan memindahkan pointer ke nilai null.

home_carousel_header.xml,

CarouselHeaderAdapter.Kt, & CarouselHeaderViewHolder.Kt XML

home_carousel_header.xml merupakan layout yang akan digunakan oleh Carousel yang terdapat pada Homepage yang ditampilkan melalui ViewGroup “RecyclerView”. Data-data yang ingin ditampilkan akan dikoneksikan ke RecyclerView melalui Adapter dimana data tersebut akan dikoneksikan ke masing-masing View terlebih dahulu melalui ViewHolder.

CarouselHeaderViewHolder.Kt merupakan class yang menghubungkan data-data dan behaviour yang akan dimuat ke dalam ViewGroup “RecyclerView” Carousel dengan posisi yang diatur secara horizontal dan bagaimana Carousel dihubungkan ke file XML. CarouselHeaderAdapter.Kt merupakan class Adapter yang melakukan koneksi pada layout UI dengan data yang dihubungkan pada ViewHolder serta behavior jika Carousel ditekan.

home_carousel_card.xml,

CarouselAdapter.Kt, & CarouselViewHolder.Kt XML

home_carousel_card.xml merupakan layout komponen-komponen UI yang akan membangun Carousel yang telah dibuat yang terdiri dari “MaterialCardView” dengan margin keseluruhan 8dp, elevasi 4dp, ketebalan outline 1dp dan warna dari Material 3 Outline. Di dalamnya, terdapat “LinearLayout” horizontal dengan padding 16dp dan orientasi child View tengah. Di dalamnya, terdapat View “ShapeableImageView” yang merupakan komponen gambar dengan gaya yang dapat diatur. Tinggi “ShapeableImageView” adalah 400dp dan lebarnya 200dp, isi gambar memotong dirinya jika memenuhi View, dan gaya View menggunakan Material 3 Corner untuk Rounded Corner.

CarouselHeaderViewHolder.Kt merupakan class yang menghubungkan data-data dan behaviour yang akan dimuat ke dalam ViewGroup “MaterialCardView” dan View “ShapeableImageView” CarouselAdapter.Kt merupakan class Adapter yang melakukan koneksi pada layout UI dengan data yang telah dihubungkan pada ViewHolder serta behavior jika Carousel ditekan.

home_item_card.xml,

ItemCardAdapter.Kt, & ItemCardViewHolder.Kt XML

home_item_card.xml merupakan layout komponen-komponen UI yang akan membangun item list yang terdiri dari “MaterialCardView” dengan margin keseluruhan 8dp, elevasi 4dp, ketebalan outline 1dp dan warna dari Material 3 Outline. Di dalamnya, terdapat “LinearLayout” horizontal dengan padding 16dp dan orientasi child View tengah. Di dalamnya, terdapat View “ShapeableImageView” dengan tinggi 100dp dan lebar 140dp, isi gambar memenuhi View, dan gaya View menggunakan Material 3 Corner untuk Rounded Corner.

Di dalamnya, terdapat “LinearLayout” vertikal dengan margin awal 16dp yang memuat View “TextView” title dengan gaya teks Material 3 Headline, “TextView” subtitle dan description dengan gaya teks Material 3 Body. Di bawahnya, terdapat “LinearLayout” horizontal dengan margin atas 24dp dan orientasi komponen view di akhir. Di dalamnya, terdapat View “MaterialButton” yang memuat tombol dengan gaya Material 3 dengan tombol pertama memiliki padding horizontal 16dp berfungsi sebagai explicit intent ke Steam Page dan tombol kedua berfungsi untuk navigasi ke Detail Page dengan gaya teks Material 3 Body dan dipisah dengan View “Space” berjarak 8dp.

ItemCardViewHolder.Kt merupakan class yang menghubungkan data-data yang akan dimuat ke dalam masing-masing view yang terdapat pada home_item_card.xml. ItemCardAdapter.Kt merupakan class Adapter yang melakukan koneksi pada layout UI dengan data-data yang telah dihubungkan pada ViewHolder serta behavior jika tombol ditekan.

MainActivity.Kt & ScrollableApp.Kt Compose

MainActivity.Kt berfungsi sebagai Activity utama dari Aplikasi Android dengan konfigurasi edge to edge, tema aplikasi menyesuaikan tema sistem Android, dan layout yang diambil dari method composable “ScrollableApp()” di file ScrollableApp.Kt. Pada file ScrollableApp.Kt, terdapat Class yang menyimpan tipe data enum dengan nama “ScrollableScreen” yang berisikan tiga konstanta yang dipakai untuk navigasi, yaitu “Home”, “Details”, dan “Settings”.

Method composable “ScrollableApp()” berfungsi untuk menampilkan layout utama Aplikasi pada Activity serta navigasi aplikasi per layar screen. Sebelumnya, terdapat inisialisasi locale yang dilakukan menggunakan ViewModelFactory. Navigasi dilakukan dengan “NavHost” dan “NavController” dengan layar default Homepage dan tiga rute navigasi yang dikoneksikan melalui method navigator “composable()”. Pada rute Homepage, terdapat inisialisasi composable “HomeScreen()” yang menginisialisasikan UI State dari ScrollableUiState, logika tombol detail yang bernavigasi ke screen Detail Page sesuai nilai indeks, logika tombol intent yang menjalankan explicit intent ke Steam Page, logika tombol setting yang bernavigasi ke screen Setting Page, dan logika tombol exit yang menutup aplikasi. Tombol intent dan detail terdapat panggilan ke ViewModel untuk logging ke Logcat.

Pada rute Detail Page, nilai indeks item akan diperbarui secara asynchronous, kemudian menginisialisasikan composable “DetailScreen()” dengan parameter “item” yang menentukan item mana yang dimuat dari data, “modifier” sebagai modifier layout, dan “onBackClick” sebagai logika tombol back. Pada rute Setting Page, terdapat inisialisasi composable “SettingScreen()” dengan parameter “modifier” sebagai modifier layout, “onBackClick” sebagai logika tombol back, “onLocaleChange” sebagai logika konfigurasi bahasa yang memanggil method pada ViewModel, dan “selectedLocale” yang menyimpan State Locale ke ScrollableUiState.

SettingScreen.Kt Compose

Method Composable “SettingScreen()” berfungsi untuk memuat layout UI pada Setting Page dengan komponen “Scaffold” untuk memuat kerangka UI yang terdiri dari Topbar orientasi tengah dengan title atau judul dan ikon navigasi yang diambil dari Material 3 yang memanggil method “onBackClick” jika ditekan. Layout dibawah Topbar terdiri dari “Column” yang memuat kolom dengan padding yang menyesuaikan komponen “Scaffold” dan jarak keseluruhan 16dp dan menyimpan UI serta dapat melakukan scroll vertikal dan menyimpan posisi scroll.

Di dalam komponen “Column”, terdapat komponen “Text” yang memuat teks pengaturan/setting dengan gaya dari Material 3 Headline dan padding vertikal 8dp. Kemudian, terdapat tiga komponen yang berfungsi untuk memuat masing-masing UI Radio Button, yaitu komponen “Row” yang memuat baris dengan orientasi tengah vertikal yang didalamnya terdapat komponen “RadioButton” yang terpilih jika nilai “selectedLocale” sesuai dengan Locale Sistem dan panggilan method untuk mengubah Locale, serta komponen “Text” yang mengasih keterangan pada tiap komponen “RadioButton”. Method preview “SettingScreenLayout()” berfungsi untuk memuat UI dari composable yang telah dibuat tanpa harus menjalankan aplikasi.

DetailScreen.Kt Compose

Method Composable “DetailScreen()” berfungsi untuk memuat layout UI pada Detail Page dengan komponen “Scaffold” untuk memuat kerangka UI yang terdiri dari Topbar orientasi tengah dengan title dengan komponen “Text” dan ikon navigasi dari Material 3 yang memanggil method “onBackClick” jika ditekan. Layout dibawahnya terdiri dari komponen kolom “Column” scrollable dengan padding menyesuaikan kerangka.

Di dalam komponen “Column”, terdapat komponen gambar “Image” yang mengambil sumber gambar dari data dengan lebar memenuhi parent dan tinggi 450dp. Di bawahnya, terdapat komponen “Column” dengan padding keseluruhan 16dp yang memuat komponen “Text” title dan subtitle dengan gaya teks Material 3 Headline. Di bawahnya, terdapat komponen “Card” yang memuat suatu tempat dengan padding atas 8dp, elevasi 4dp, bentuk pojok melengkung sebesar 8dp, outline setebal 1dp dengan gaya Material 3 outline. Di dalam komponen “Card”, terdapat komponen “Column” dengan padding 8dp yang memuat komponen “Text” description dengan gaya Material 3 Title dan “Text” detail dengan gaya Material 3 Body. Method preview “DetailScreenPreview()” berfungsi untuk memuat UI dari composable yang telah dibuat tanpa harus menjalankan aplikasi.

HomeScreen.Kt Compose

Method Composable “HomeAppBar()” berfungsi untuk memuat layout Topbar UI yang terdiri dari Topbar biasa yang terdiri dari sub komponen title, ikon navigasi exit, dan ikon navigasi actions yang digunakan untuk berpindah ke Setting Page. Method Composable “HomeScreen()” berfungsi untuk memuat layout UI Homepage menggunakan komponen “Scaffold” sebagai kerangka yang terdiri dari Topbar yang diambil dari method composable “HomeAppBar()” dengan parameter “OnSettingClick” untuk berpindah ke setting dan parameter “OnExit” untuk keluar dari aplikasi.

Isi dari “HomeScreen()” adalah komponen “LazyColumn” dengan padding menyesuaikan kerangka “Scaffold” dan padding awal 8dp yang memuat UI scrollable yang hanya terlihat pada layar. “LazyColumn” digunakan untuk memuat data-data yang dimuat pada Carousel yang didefinisikan pada composable “ItemCarousel()” serta logika tombol detail. Selain itu, juga terdapat pemuatan data-data berbasis indeks ke item list yang didefinisikan pada composable “ItemCard()” dengan logika tombol detail serta tombol dengan explicit intent.

Composable “ItemCard()” berfungsi untuk memuat UI item list yang terdiri dari komponen “Card” dengan padding 8dp, ukuran Material 3 Medium, elevasi 4dp, dan border atau outline setebal 1dp dengan warna Material 3 outline. Di dalamnya, terdapat komponen “Row” dengan padding keseluruhan 16dp yang berisikan komponen gambar “Image” yang mengambil sumbernya dari data pada DataSource.Kt, isi gambar memenuhi View, dan modifier tinggi 140dp dan lebar 100dp serta ukuran pojok melengkung sebesar 16dp.

Di sebelah komponen “Image”, terdapat komponen “Column” dengan padding awal 16dp dan komponen mengambil setengah dari space parent View. Di dalamnya, terdapat komponen “Text” yang memuat teks title dengan gaya material 3 Headline, teks subtitle dan description dengan gaya material 3 body. Di bawahnya, terdapat komponen “Row” dengan padding atas 24dp dan orientasi child View berada di akhir. Di dalamnya, terdapat komponen tombol “Button” dengan tombol pertama memiliki padding horizontal 16dp dan kedua teks pada tombol memiliki gaya Material 3 body. Kedua tombol dipisah dengan komponen “Spacer” dengan jarak 8dp.

Composable “CarouselCard()” berfungsi untuk memuat UI isi Carousel yang terdiri dari komponen “Card” dengan padding 8dp, ukuran Material 3 Medium, elevasi 4dp, border atau outline setebal 1dp dengan warna Material 3 outline, warna Kartu atau “Card” berupa Material 3 Surface Container, dan memanggil “onDetailClick” jika ditekan. Di dalamnya, terdapat komponen “Row” dengan padding keseluruhan 16dp dan terdapat even spacing horizontal agar gambar berada di tengah. Komponen “Image” di dalam “Row” mengambil data dari DataSource.Kt, isi gambar memotong dirinya jika memenuhi View, dan modifier tinggi 200dp, lebar 400dp, dan lengkungan sebesar 16dp.

Composable “ItemCarousel()” berfungsi untuk memuat UI Carousel melalui komponen “LazyRow” yang menyimpan scroll state dan berhenti di item terdekat dalam scrolling. Di dalamnya, terdapat pemuatan data-data berdasarkan indeks ke dalam composable “CarouselCard()” beserta logika klik navigasi ke detail serta lebar child Composable mengambil penuh lebar View. Method preview “HomeScreenPreview()” berfungsi untuk memuat UI dari composable yang telah dibuat tanpa harus menjalankan aplikasi.

Debugger

Debugger pada Android Studio adalah Tools yang essential untuk pengembangan Aplikasi. Debugger digunakan untuk melacak dan memperbaiki bug pada program yang dikembangkan serta menguji program yang sedang dikembangkan. Debugger berfungsi untuk memeriksa nilai variabel dan melacak alur eksekusi program agar developer dapat memahami permasalahan pada kode sehingga dapat mengatur program secara terkontrol.

Debugger yang disediakan oleh Android Studio adalah debugger dinamis yang menganalisa kode program saat runtime. Debugger dinamis memiliki beberapa tools seperti Breakpoint yang menghentikan sementara program (pause) saat baris kode tertentu dijalankan kemudian Android Studio menampilkan nilai variabel dan kondisi memori sesuai dengan kondisi Breakpoint.

Step Into, Step Over, dan Step Out

Dalam debugging, terdapat fitur-fitur berupa Step Into, Step Over, dan Step Out. Fitur Step Into digunakan untuk masuk ke method pada baris kode yang dipanggil. Step Over menjalankan baris kode yang sedang ditunjuk kemudian lanjut tanpa memasuki method apapun. Step Out menjalankan method yang ditunjuk hingga selesai dan kembali ke baris kode pemanggil.

D. Essay

1. Jelaskan Application class dalam arsitektur aplikasi Android dan fungsinya

Jawab:

Application Class dalam arsitektur aplikasi Android adalah base class yang memuat seluruh komponen lain seperti Activity dan Service dan berperan sebagai titik masuk siklus hidup aplikasi yang menyediakan tempat yang tersentralisasi untuk inisialisasi dan manajemen Global State aplikasi [1]. Application Class atau subclassnya diinstansiasi terlebih dahulu sebelum Class lain ketika proses pada aplikasi atau package dibuat. Application Class berfungsi sebagai inisialisasi global variabel atau global state seperti library atau framework tertentu sebelum Activity utama ditampilkan sehingga tidak perlu melakukan inisialisasi pada tiap komponen-komponen yang telah dibuat [2].

E. Daftar Pustaka

- [1] S. Yeole, "Application Class in Android," Medium, 21 Mei 2023. [Online]. Available: <https://medium.com/@sdycode/application-class-in-android-474b55507f6f>. [Diakses 12 Mei 2026].
- [2] C. Clement, "Understanding the Android Application Class," codepath.org, 3 Maret 2021. [Online]. Available: <https://guides.codepath.org/android/Understanding-the-Android-Application-Class>. [Diakses 12 Mei 2026].

Tautan GIT

Berikut adalah tautan GIT untuk Praktikum Pemrograman Mobile

<https://github.com/Arz-zrs/Pemrograman-Mobile>